FIAP ON

IOS

WEBKIT E RECURSOS MULTIMÍDIA

8

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 – Criando *IBAction* para o botão 7
Figura 2 – Adicionando permissões no Info.plist 11
Figura 3 – Vinculando a nova cena à Tab Bar Controller 12
Figura 4 – Tela Map 13
Figura 5 – Preparando Text Field e Map Kit View 14
Figura 6 – Preparando Outlet e Action dos Componentes 15
Figura 7 – Adicionando entrada de uso de localização 17
Figura 8 – Mapa mostrando localização do usuário 19
Figura 9 – Rota traçada.....27
Figura 10 – Adicionando WebKit View 28
Figura 11 – Classe e cena da *Web View* 29
Figura 12 – Criando segue *webSegue* 29
Figura 13 – Permitindo o uso de sites não seguros 31
Figura 14 – Entrando no programa de desenvolvedores Apple 35
Figura 15 – Associando conta e gerando certificado de desenvolvimento 36
Figura 16 – Certificado gerado no portal 37
Figura 17 – Iniciando criação de certificado de produção 37
Figura 18 – Assistente de Certificado.....38
Figura 19 – Registrando App ID 39
Figura 20 – App ID registrado 40
Figura 21 – Criando novo app na App Store 41
Figura 22 – Preenchendo informações do App 42
Figura 23 – Preços e disponibilidade 42
Figura 24 – Preenchendo informações da versão 43
Figura 25 – Escolhendo Generic iOS Device 44
Figura 26 – Iniciando processo de envio do App.....45
Figura 27 – Upload concluído com sucesso.....45
Figura 28 – Atividade 46
Figura 29 – Selecionando compilação 47

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Alerta de escolha de fonte de imagem.....8
Código-fonte 2 – Implementando *selectPicture*.....8
Código-fonte 3 – Implementando protocolos.....10
Código-fonte 4 – Importando MapKit, inserindo Outlets de mapa e texto e Action de Button.....15
Código-fonte 5 – Chamando API na classe ViewController 18
Código-fonte 6 – Realizando a busca de pontos de interesse 21
Código-fonte 7 – Adicionando *annotations* no mapa.....22
Código-fonte 8 – Implementando *MKMapViewDelegate*.....23
Código-fonte 9 – Definindo *delegate* do *Map View* 24
Código-fonte 10 – Implementando método que mostra a rota 26
Código-fonte 11 – Implementando *renderer* para a rota 26
Código-fonte 12 – Chamando segue *webSegue*.....30
Código-fonte 13 – Enviando URL para nova tela 30
Código-fonte 14 – Classe *SiteViewController*.....31
Código-fonte 15 – Objetos *AVPlayer* e *AVPlayerViewController*.....32
Código-fonte 16 – Preparando *moviePlayer* e *moviePlayerViewController*.....33
Código-fonte 17 – Chamando *prepareVideo*.....34
Código-fonte 18 – Apresentando e tocando o trailer 34

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 O ECOSSISTEMA DE SENSORES E MULTIMÍDIA 5
1.1 Recuperando fotos com UIImagePickerController 5
1.1.1 UIImagePickerController 5
1.1.2 UIImagePickerControllerDelegate e UINavigationControllerDelegate 9
1.2 Utilizando mapas: *MapKit* 11
1.3 Localização do usuário com *CoreLocation* 16
1.3.1 CLLocationManager 17
1.4 Pesquisando pontos de interesse 19
2 ADICIONANDO *ANNOTATIONS* NO MAPA 21
2.1 Traçando rotas 22
2.2 Usando *WebView* em seus Apps 27
2.3 Vídeos: AVPlayer, AVPlayerViewController 32
2.4 Publicando apps na App Store 34
CONCLUSÃO 48
REFERÊNCIAS 49

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 O ECOSSISTEMA DE SENSORES E MULTIMÍDIA

Neste capítulo, vamos conhecer sobre uso de mapas, localização, como traçar rotas e, por fim, como publicar seu aplicativo na Apple Store.

Agora que você já sabe manipular uma série de elementos dentro do seu aplicativo iOS, chegou a hora de falarmos um pouco mais sobre diferentes formatos de mídia. Aqui, abordaremos o conceito de *WebKit*, além dos recursos multimídia que tornarão seu aplicativo ainda mais intuitivo e dinâmico.

1.1 Recuperando fotos com UIImagePickerController

Nosso aplicativo já está realizando a tarefa de gravar filmes em um banco de dados local, utilizando o recurso *Core Data*, porém, uma das partes que está faltando é a gravação dos pôsteres dos filmes. Quando precisamos solicitar ao usuário que escolha uma foto para sua aplicação, fazemos uso de uma classe que foi projetada para fornecer todos os controles necessários para essa ação. Essa classe é a ***UIImagePickerController***.

1.1.1 UIImagePickerController

Esta classe gerencia a interação do usuário com sua biblioteca de fotos, bem como permite o acesso à câmera do dispositivo. Por meio de seu *delegate*, somos notificados quando o usuário escolheu ou tirou a foto, ou até mesmo se cancelou o processo.

Caso tenha escolhido/tirado uma foto, temos acesso a essa mídia, podendo utilizá-la imediatamente. Um detalhe importante sobre o *Image Picker Controller* é que ele suporta apenas o modo *portrait*, ou seja, somente conseguiremos tirar fotos retrato.

Para controlarmos se o usuário irá recuperar fotos da sua biblioteca ou se utilizará a câmera do *device*, definimos a propriedade ***sourceType***, cujos possíveis valores são:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- ***photoLibrary***: Define a biblioteca de fotos como sendo a fonte do *UIImagePickerController*.
- ***camera***: Define a fonte como a câmera do *device*. Você pode definir qual câmera (*front* ou *rear*) por meio da propriedade *cameraDevice*.
- ***savedPhotosAlbum***: Define o álbum de fotos da câmera como fonte. Caso o *device* não tenha câmera, ele utiliza a biblioteca de fotos no lugar.

Para utilizarmos o *UIImagePickerController*, faremos o seguinte:

- Verificaremos se a câmera está disponível (por meio do método de classe *isSourceTypeAvailable*(), passando **.camera** como parâmetro) para oferecermos essa opção ao usuário, juntamente com as opções de biblioteca de fotos e álbum de fotos.
- Criaremos o objeto do tipo *UIImagePickerController*, definindo seu *sourceType* baseado na escolha do usuário.
- Definiremos seu *delegate* como sendo a própria classe na qual foi criado.
- Apresentamos ao usuário por meio do método *present* da *UIViewController*.

No *storyboard*, vá até a tela de cadastro de filmes e crie uma *IBAction* para o botão de adicionar pôster. Dê o nome de ***addPoster*** para a *action* e no *Type,* defina como ***UIButton***. Veja que também é possível criar o IBAction pelo Document Outline, selecionando o “Adicionar Pôster” na lista de componentes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

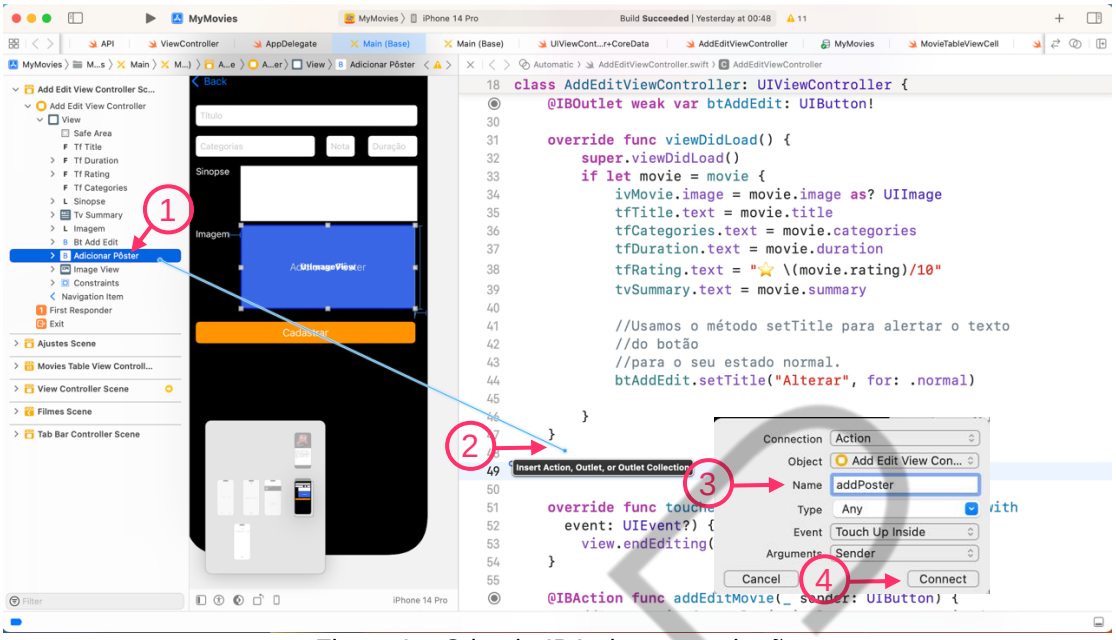

Figura 1 – Criando *IBAction* para o botão
Fonte: Elaborado pelo autor (2023)

Vamos implementar o código dessas *actions* conforme o código “Alerta de escolha de fonte de imagem”. Aqui, estamos criando um *Action Sheet*, no qual o usuário escolherá de onde pretende tirar a foto do pôster do filme. A verificação da existência da câmera é necessária, pois, quando rodamos o App no simulador, não temos acesso à câmera, sendo assim, não vamos mostrar o respectivo botão no alerta.

```
@IBAction func addPoster(_ sender: UIButton) {
    //Criando alerta do tipo actionSheet
    let alert = UIAlertController(title: "Selecionar pôster",
message: "De onde você quer escolher o pôster?",
preferredStyle: .actionSheet)

    //Verificamos se a câmera está disponível no device.
    //Obs.: Não temos acesso à câmera pelo simulador.
    if UIImagePickerController.isSourceTypeAvailable(.camera)
{
        let cameraAction = UIAlertAction(title: "Câmera",
style: .default) { (action) in
            self.selectPicture(source: .camera)
        }
        alert.addAction(cameraAction)
    }

    //Criando as actions para abrir a Biblioteca de Fotos
    //e o Álbum de fotos
    let libraryAction = UIAlertAction(title: "Biblioteca de
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Fotos", style: .default) { (action) in
        self.selectPicture(source: .photoLibrary)
    }
    alert.addAction(libraryAction)
    let photosAction = UIAlertAction(title: "Álbum de Fotos",
style: .default) { (action) in
        self.selectPicture(source: .savedPhotosAlbum)
    }
    alert.addAction(photosAction)

    let cancelAction = UIAlertAction(title: "Cancelar", style:
.cancel, handler: nil)
    alert.addAction(cancelAction)

    //Mostrando o alerta
    present(alert, animated: true, completion: nil)
}
```

Código-fonte 1 – Alerta de escolha de fonte de imagem
Fonte: Elaborado pelo autor (2019)

O código em questão faz uso do método *selectPicture*, que será o responsável por montar e apresentar o *Alert Picker Controller* baseado na escolha do usuário. Para que seja tirado os erros que aparecem em todas as linhas que chamamos o método *selectPicture*, vamos criar esse método conforme o código-fonte "Implementando *selectPicture*":

```
func selectPicture(source: UIImagePickerController.SourceType)
{
    //Criando objeto UIImaePickerController
    let imagePicker = UIImagePickerController()

    //Definindo sourceType baseado no parâmetro
    imagePicker.sourceType = source

    //Definimos esta classe como sua delegate
    imagePicker.delegate = self

    //Apresentando ao usuário
    present(imagePicker, animated: true, completion: nil)
}
```

Código-fonte 2 – Implementando *selectPicture*
Fonte: Elaborado pelo autor (2019)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.2 UIImagePickerControllerDelegate e UINavigationControllerDelegate

Você irá notar que ainda existe um erro no self do método *selectPicture*. Para que possamos utilizar nossa classe como *delegate* da *UIImagePickerController*, precisaremos que ela esteja em conformidade com o protocolo *UIImagePickerControllerDelegate*. Como a classe *UIImagePickerController* é subclasse de *UINavigationController*, também precisamos implementar o protocolo *UINavigationControllerDelegate*. Todos os métodos desses protocolos são opcionais, mas, para recuperarmos a imagem selecionada, teremos que implementar o método ***imagePickerController(_ picker: UIImagePickerController, didFinishPickingMediaWithInfo info: [String : Any])***. Esse método nos fornece um dicionário do qual podemos recuperar a imagem selecionada. Implemente os protocolos e o método, conforme o código-fonte "Implementando protocolos", e atente para as explicações.

```
extension AddEditViewController:
UIImagePickerControllerDelegate,
UINavigationControllerDelegate {
    func imagePickerController(_ picker:
UIImagePickerController, didFinishPickingMediaWithInfo info:
[UIImagePickerController.InfoKey: Any]) {
        //Recuperando de forma segura a imagem através da
        //chave UIImagePickerController.InfoKey.originalImage
        if let image =
info[UIImagePickerController.InfoKey.originalImage] as?
UIImage {

            //Vamos recuperar a largura e altura originais
            //para definirmos a proporção da imagem e assim
            //podermos reduzi-la, deixando com 800 pixels de
            //tamanho no seu lado maior.
            let aspectRatio = image.size.width /
image.size.height
            var smallSize: CGSize
            if aspectRatio > 1 {
                smallSize = CGSize(width: 800, height:
800/aspectRatio)
            } else {
                smallSize = CGSize(width: 800*aspectRatio,
height: 800)
            }

            //Os códigos abaixo são necessários para criarmos
            //uma versão reduzida da imagem
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            UIGraphicsBeginImageContext(smallSize)
            image.draw(in: CGRect(x: 0, y: 0, width:
smallSize.width, height: smallSize.height))
            let smallImage =
UIGraphicsGetImageFromCurrentImageContext()
            UIGraphicsEndImageContext()

            //Vamos atribuir a imagem pequena à nossa
            //Image View
            ivMovie.image = smallImage

            //Aqui, fechamos o Image Picker
            dismiss(animated: true, completion: nil)
        }
    }
}
```

Código-fonte 3 – Implementando protocolos
Fonte: Elaborado pelo autor (2019)

Obrigatoriamente, para que possamos ter acesso à câmera do usuário, bem como à sua biblioteca de fotos, precisamos pedir sua permissão. Essa solicitação é feita automaticamente pela *UIImagePickerController*, porém devemos inserir duas entradas no arquivo **info.plist**: uma para a câmera e outra para a biblioteca de fotos. Abra o arquivo info.plist e implemente as entradas conforme a figura "Adicionando permissões no Info.plist". Você configura o info.plist clicando em **Info**, depois no **"+"** que está depois da frase "*Aplication Scene Manifest*", em seguida comece a digitar "*Privacy – Camera Usage Description*" até que apareça este item. Em seguida, clique novamente no "+" em frente ao novo item que acabou de ser adicionado e digite o próximo item como: "*Privacy – Photo Library Usage Description*".

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

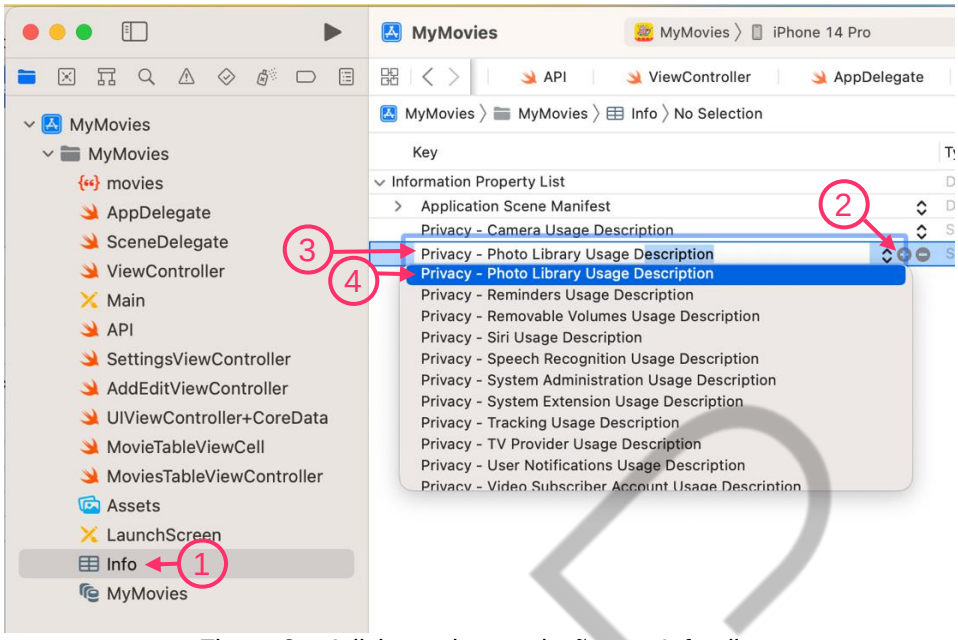

Figura 2 – Adicionando permissões no Info.plist
Fonte: Elaborado pelo autor (2023)

Rode seu aplicativo, cadastre um filme e veja que agora é possível adicionar a imagem do pôster do filme, recuperando-a da câmera ou da biblioteca/álbum de fotos do aparelho. Como a câmera só funciona em aparelho físico, caso você esteja utilizando o simulador, adicione uma imagem clicando em álbum de fotos e escolhendo uma imagem qualquer sugerida pelo simulador.

1.2 Utilizando mapas: *MapKit*

Quando precisamos utilizar mapas em aplicativos iOS, utilizamos um componente específico para esse fim, chamado *Map Kit View* (cuja classe é ***MKMapView***). Esse componente, assim como os demais recursos que utilizamos para trabalhar com mapas e com a localização do usuário, encontram-se no *framework* ***MapKit***.

Podemos utilizar mapas com vista padrão (*Standard*), satélite (*Satellite*) ou híbrida (*Hybrid*). Você pode alterar o tipo de mapa via código, ou por meio da propriedade ***Type*** via *Storyboard*.

Podemos, também, definir se o nosso mapa poderá ser rotacionado, aplicar zoom, scroll, mostrar os pontos de interesse etc., por meio das respectivas propriedades no *Storyboard*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No nosso projeto, iremos adicionar uma nova *View Controller* dentro da *Tab Bar Controller*. Essa nova tela irá mostrar um mapa e a opção do usuário de pesquisar por pontos de interesse. A ideia é que ele possa pesquisar por cinemas perto da sua localização.

Adicione uma nova *View Controller*, faça com que ela seja controlada pela *Tab Bar Controller* e posicione seu *Tab Bar Item* no meio, entre Filmes e Ajustes. Caso não lembre como adicionar a nova View Controller no TabBar, clique na *Tab Bar Controller* e o vincule arrastando uma ligação da Tab Bar Controller até a cena recém-criada e escolha a opção view controllers.

Figura 3 – Vinculando a nova cena à Tab Bar Controller
Fonte: Elaborado pelo autor (2022)

Crie uma classe chamada ***MapViewController*** e vincule essa classe à nova cena criada pelo Identity Inspector. Altere o *Tab Bar Item* dessa tela para que tenha o título **Mapa** e a imagem ***map***. Seu *storyboard* ficará como mostra a Figura “Tela Map”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

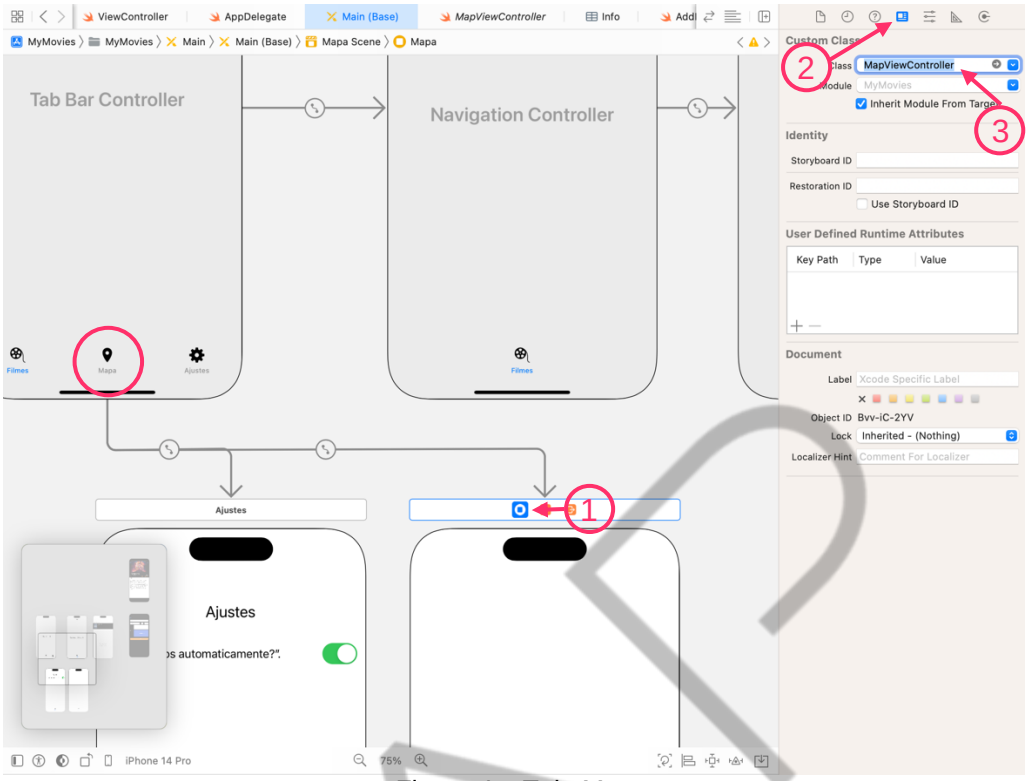

Figura 4 – Tela Map
Fonte: Elaborado pelo autor (2023)

Vamos agora adicionar dois componentes: o *Map Kit View* e o componente *Text Field*. No nosso app, ele será utilizado para que o usuário pesquise por cinemas (ou qualquer outro ponto de interesse).

Adicione-o à tela de modo que o *Text Field* fique no topo (logo abaixo da status bar) e o mapa fique imediatamente abaixo dele, ocupe todo o restante da *view*. Selecione o *Text Field* e defina seu *placeholder* como "**Entre com o ponto de interesse**".

Adicione um *Button* ao lado do *Text Field,* digite **Buscar** no texto do botão, marque as ***constrains***, **topo**, **direita** e ***Width***.

Selecione os componentes *Text Field* e *Map View* e crie, ao mesmo tempo, as ***constraints*** de **top**, ***bottom***, ***leading*** e ***trailing*** certificando-se de que todas estejam com relação à *Safe Area* e com valores marcados **como 0**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

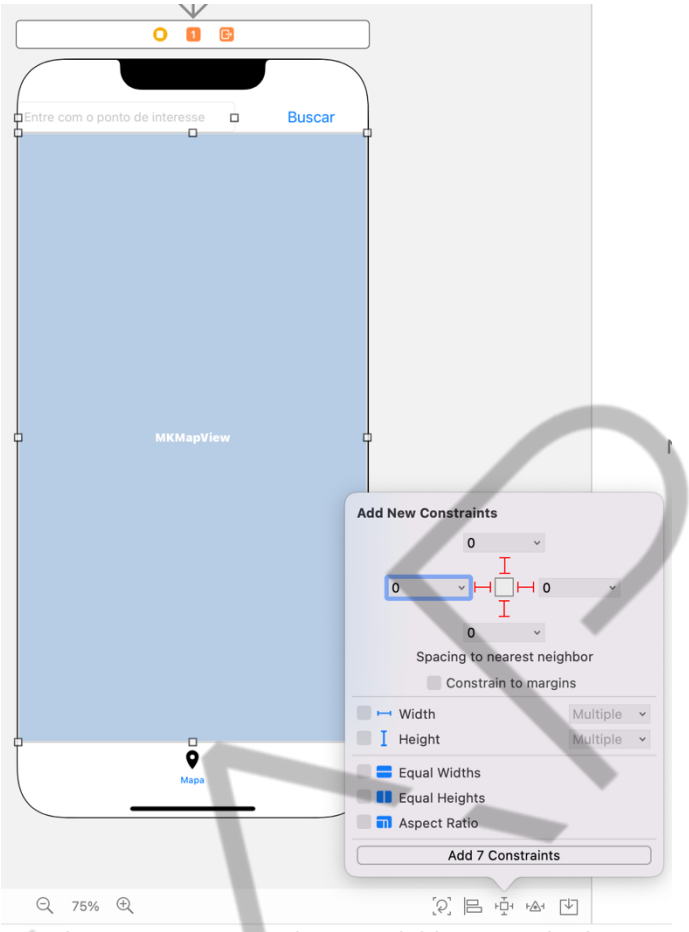

Figura 5 – Preparando Text Field e Map Kit View
Fonte: Elaborado pelo autor (2023)

Depois, crie uma *outlet* para **o *Map Kit View*** com o nome ***mapView*** e uma para o ***Text Field*** com o nome ***textField***. Além disso, crie um ***Action*** para o botão como **searchButton**. Você terá que importar **o *framework MapKit*** para poder utilizar o mapa nessa cena e retirar o erro que ocorre no outlet caso tenha criado o outlet antes de ter feito o import.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

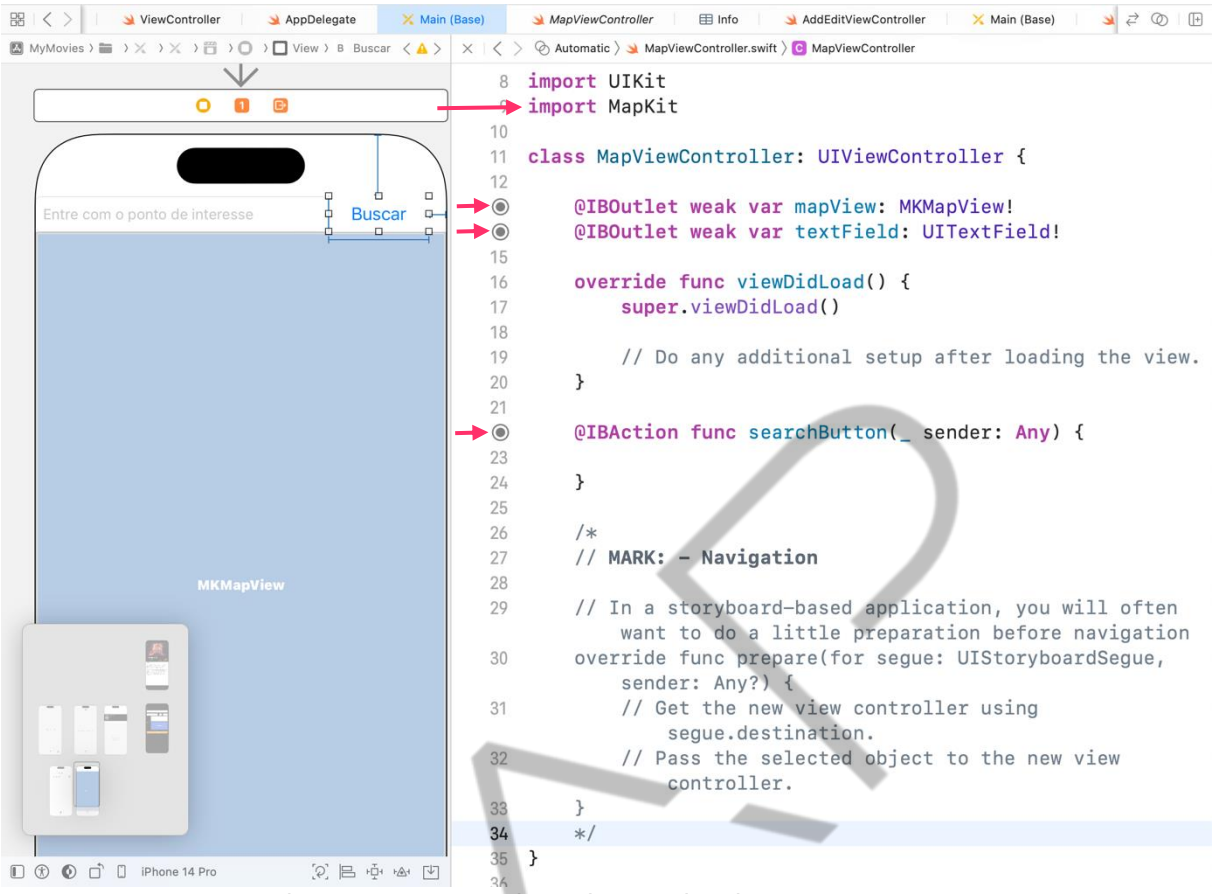

Figura 6 – Preparando Outlet e Action dos Componentes
Fonte: Elaborado pelo autor (2023)

```
import UIKit
import MapKit

class MapViewController: UIViewController {

    @IBOutlet weak var mapView: MKMapView!
    @IBOutlet weak var textField: UITextField!


    override func viewDidLoad() {
        super.viewDidLoad()

    }


     @IBAction func searchButton(_ sender: Any) {

     }
}
```

Código-fonte 4 – Importando MapKit, inserindo Outlets de mapa e texto e Action de Button
Fonte: Elaborado pelo autor (2023)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Rode o aplicativo e acesse a tela de mapa. Veja que agora nosso app está com um mapa que pode ser manipulado livremente pelo usuário.

1.3 Localização do usuário com *CoreLocation*

Você deve ter notado que, quando visualizamos o mapa, ele mostra a região do Brasil. A ideia do nosso aplicativo é que o usuário possa utilizar o mapa para pesquisar por pontos de interesse perto da sua localização, principalmente, que ele possa pesquisar por cinemas próximos à região onde ele está, e para isso temos que recuperar a localização do usuário.

CoreLocation é o *framework* que nos dá acesso aos dados de localização do usuário e nos fornece uma série de classes responsáveis por trabalhar essa informação, como, por exemplo, monitorar a localização do usuário. Esse *framework* já é importado automaticamente quando importamos o *MapKit*.

Para que possamos ter acesso à localização do usuário, precisamos solicitar essa autorização e, para isso, é necessário inserir no **info.plist** uma entrada na qual iremos determinar a mensagem que será apresentada ao usuário quando essa autorização for solicitada.

Podemos solicitar a autorização para uso da localização somente quando o usuário estiver usando o App (***Privacy – Location When In Use Usage Description***) ou também quando o App estiver em background (***Privacy – Location Always Usage Description***). No nosso app, iremos solicitar autorização apenas para quando o App estiver em uso.

Abra o arquivo info.plist e adicione a entrada ***Privacy – Location When In Use Usage Description*** e utilize a mensagem **Precisamos acessar a sua localização**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

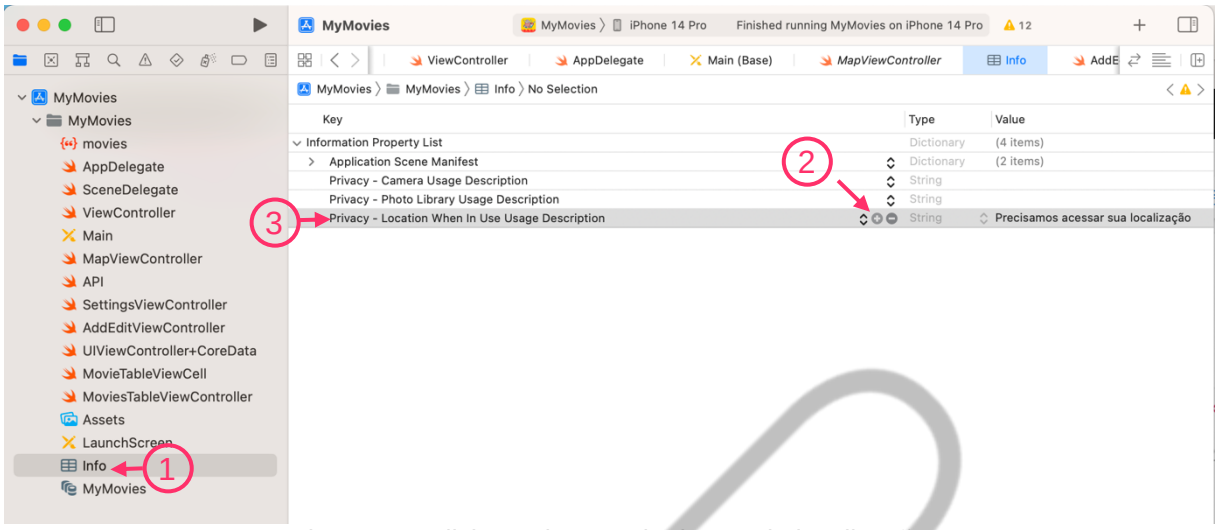

Figura 7 – Adicionando entrada de uso de localização
Fonte: Elaborado pelo autor (2023)

1.3.1 CLLocationManager

Para manipularmos a localização do usuário, utilizamos a classe ***CLLocationManger***. É com ela que conseguimos monitorar onde o usuário se encontra, sendo a responsável por pedir ao usuário a autorização de acesso à sua localização.

Vamos criar um objeto do tipo *CLLocationManager* e criar um método que irá solicitar ao usuário o acesso à sua localização. Outra coisa que vamos fazer é solicitar ao *Map View* que ele monitore a localização do usuário. Ao fazer isso, o mapa ficará sempre centralizado onde o usuário estiver. Isso é muito importante para o nosso App, pois queremos que o usuário pesquise por cinemas perto de onde ele estiver.

Deixe sua classe *MapViewController* conforme o código-fonte “Chamando API na classe ViewController”. Preste atenção às explicações contidas no código.

```
import UIKit
import MapKit

class MapViewController: UIViewController {

    @IBOutlet weak var mapView: MKMapView!
    @IBOutlet weak var textField: UITextField!
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    //Criando objeto CLLocationManager
    var locationManager = CLLocationManager()

    override func viewDidLoad() {
        super.viewDidLoad()

        //Pedimos que o mapa mostre a localização do usuário
        mapView.showsUserLocation = true

        //Através da propriedade userTrackingMode, definimos
que o mapa irá
        //rastrear a localização do usuário, centralizando
onde ele estiver
        mapView.userTrackingMode = .follow

        requestAuthorization()
    }

    //Criamos o método abaixo para solicitar a autorização do
usuário
    //para acesso à sua localização
    func requestAuthorization() {
        //Definindo precisão da localização. A constante
kCLLocationAccuracyBest
        //fornece a maior precisão possível
        locationManager.desiredAccuracy =
kCLLocationAccuracyBest

        //Método que solicita a autorização
        locationManager.requestWhenInUseAuthorization()
    }
}
```

Código-fonte 5 – Chamando API na classe ViewController
Fonte: Elaborado pelo autor (2023)

Para que você possa simular a sua localização no mapa dentro do simulador, rode o aplicativo e, quando o simulador estiver rodando o app, certifique-se de que ele esteja em foco e vá ao menu ***Features -> Location -> Custom Location...***. Na janela que aparece, você pode colocar a latitude e a longitude do local que deseja simular. Coloque a latitude e a longitude de onde você está e pronto, o simulador irá simular a sua localização no mapa. Experimente essa latitude: 41,890251 com a longitude: 12,492373. Reconheceu o lugar? Tente outra, agora em SP, latitude - 23,61 e longitute - 46,6671.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 8 – Mapa mostrando localização do usuário
Fonte: Elaborado pelo autor (2023)

1.4 Pesquisando pontos de interesse

Precisamos, agora, permitir que nosso usuário pesquise por pontos de interesse, e essa pesquisa será feita ao clicar no botão **Buscar** após digitar o ponto de interesse no *Text Field*.

Precisaremos fazer uso de algumas classes para realizar essa pesquisa. A primeira delas é a classe ***MKLocalSearchRequest***, que é responsável pela configuração da pesquisa que será realizada. Nela, definimos o texto que será pesquisado, por meio da propriedade ***naturalLanguageQuery***, e qual região do mapa será utilizada para a pesquisa, por meio da propriedade ***region***. A pesquisa é feita usando linguagem natural, ou seja, se pesquisarmos café, o sistema entenderá que estamos procurando por cafeterias e não por pés de café.

Depois de produzido o objeto de *request*, é necessário criar o objeto que fará efetivamente a pesquisa. A classe responsável por isso é a ***MKLocalSearch***.

Presente nesta classe está o *método* ***start(completionHandler: @escaping MKLocalSearchCompletionHandler)***, que é o método que deve ser chamado para que a pesquisa seja realizada. O parâmetro ***MKLocalSearchCompletionHandler***, presente nesse método, é uma *closure* que nos retorna um objeto de erro, caso

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

algum tenha ocorrido, e um objeto de resposta, que contém todos os dados pesquisados.

Implemente o código para realizar a pesquisa do ponto de interesse dentro do método ***searchButton***. Seu método ficará como mostra o código-fonte "Realizando a busca de pontos de interesse".

```
@IBAction func searchButton(_ sender: Any) {

    textField.resignFirstResponder()

    //Objeto que configura os parâmetros da pesquisa
    let request = MKLocalSearch.Request()

    //A propriedade naturalLanguageQuery contém o texto que
    //será utilizado na busca
    request.naturalLanguageQuery = textField.text
    print(textField.text!)

    //Definimos a região que será usada para buscar os pontos
    //de interesse
    request.region = mapView.region

    //Aqui, criamos o objeto que realizará a busca
    let search = MKLocalSearch(request: request)

    //Através do método start realizamos a busca
    search.start { (response, error) in
      if error == nil {
        //Caso não tenha erro, recuperamos os elementos
        //solicitados
        guard let response = response else {return}

        //O objeto response possui uma propriedade
        //chamada mapItems contendo todos os locais
        //encontrados para a pesquisa.
        for item in response.mapItems {

          //Caso o local tenha um nome, imprimimos no
          //console
          if let name = item.name {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                print("-", name)
            }
        }
    }
  }
}
```

Código-fonte 6 – Realizando a busca de pontos de interesse
Fonte: Elaborado pelo autor (2021)

Execute o aplicativo, acesse o mapa e pesquise por **Cinema**. Caso tenha informado a latitude e longitude de onde você está, verá que no console irá aparecer a lista de todos os cinemas próximos à sua localização.

2 ADICIONANDO *ANNOTATIONS* NO MAPA

Agora que nosso aplicativo consegue pesquisar por pontos de interesse, precisamos mostrar esses locais no mapa. Adicionamos pontos de interesse em um mapa por meio de *annotations*, que são os objetos que representam os alfinetes que vemos em mapas quando marcamos uma localização.

A classe que representa uma *annotation* padrão é a ***MKPointAnnotation***, e é necessário possuir a latitude e a longitude do ponto para adicioná-la ao mapa. Além das coordenadas, que são criadas utilizando a *struct* ***CLLocationCoordinate2D***, podemos também definir um título e um subtítulo a uma *annotation*.

Essas informações serão retornadas do ponto de interesse recuperado pela pesquisa. Cada ponto de interesse que é trazido é representado pela classe ***MKMapItem***, e é dela que extraímos as coordenadas e quaisquer outras informações relevantes àquele ponto. No nosso exemplo, iremos recuperar o nome do estabelecimento, que será o título da *annotation*, e a URL, que será o subtítulo.

Modifique a implementação do método ***start*** do objeto ***search*** para que fique conforme o código-fonte “Adicionando annotations no mapa”.

```
search.start { (response, error) in
    if error == nil {
        //Caso não tenha erro, recuperamos os elementos
        //solicitados
        guard let response = response else {return}
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        //O objeto response possui uma propriedade chamada
        //mapItems contendo todos os locais encontrados para
        //a pesquisa.

        //Antes de adicionar novas, removemos todas as
        //annotations presentes no mapa

self.mapView.removeAnnotations(self.mapView.annotations)

        for item in response.mapItems {
            //Criando uma annotation
            let annotation = MKPointAnnotation()

            //Definindo sua coordanada, título e subtítulo
            annotation.coordinate = item.placemark.coordinate
            annotation.title = item.name
            annotation.subtitle = item.url?.absoluteString

            //Adicionando annotation no mapa
            self.mapView.addAnnotation(annotation)

        }
    }
}
```

Código-fonte 7 – Adicionando *annotations* no mapa
Fonte: Elaborado pelo autor (2019)

2.1 Traçando rotas

Vamos permitir que nosso usuário possa traçar uma rota de onde ele está até o ponto de interesse selecionado, além de permitir que ele visite o site do estabelecimento, caso ele exista. Isto será feito quando a *annotation* for selecionada no mapa.

Quando vemos uma *annotation* no mapa, o que estamos vendo, na verdade, é um objeto que serve como *view* dessa *annotation*, e sempre que essa view for selecionada, o método ***mapView(_ mapView: MKMapView, didSelect view: MKAnnotationView)*** será disparado. Esse método existe no protocolo ***MKMapViewDelegate***, e é disparado pelo mapa em seu *delegate*.

Para implementarmos esse método, precisamos definir nossa classe como *delegate* do mapa e implementar o protocolo *MKMapViewDelegate*. Implemente

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

esse protocolo, via *extension*, juntamente com o método mencionado acima, conforme o código “Implementando MKMapViewDelegate”.

```
extension MapViewController: MKMapViewDelegate {

    //Método chamado quando uma Annotation View é selecionada
    func mapView(_ mapView: MKMapView, didSelect view:
MKAnnotationView) {

        //Recuperando annotation e nome
        guard let annotation = view.annotation, let name =
annotation.title else {return}

        let alert = UIAlertController(title: name, message: "O
que deseja fazer?", preferredStyle: .actionSheet)

        //Se houver subtitle, adiciona action para ir ao site
        if let subtitle = annotation.subtitle, let url =
subtitle {
            let urlAction = UIAlertAction(title: "Acessar
site", style: .default) { (action) in
                //Ir para o site
            }
            alert.addAction(urlAction)
        }

        //Action para traçar a rota
        let routeAction = UIAlertAction(title: "Traçar rota",
style: .default) { (action) in

            //Chamando método que irá mostrar a rota
            self.showRoute(to: annotation.coordinate)
        }
        alert.addAction(routeAction)
        let cancelAction = UIAlertAction(title: "Cancelar",
style: .cancel, handler: nil)
        alert.addAction(cancelAction)
        present(alert, animated: true, completion: nil)
    }
}
```

Código-fonte 8 – Implementando *MKMapViewDelegate*
Fonte: Elaborado pelo autor (2019)

Você deve ter notado a chamada para o método ***showRoute*** no código “Implementando *MKMapViewDelegate”*. Esse é um método que nós iremos criar mais tarde para recuperar a rota.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para que o código funcione, precisamos definir nossa classe como *delegate* do *Map View*. Para isso, implemente dentro do método *viewDidLoad* o código ***mapView.delegate = self***. No final, o método viewDidLoad ficará como mostra o código-fonte "Definindo *delegate* do *Map View*".

```
override func viewDidLoad() {
    super.viewDidLoad()

    //Pedimos que o mapa mostre a localização do usuário
    mapView.showsUserLocation = true

    //Através da propriedade userTrackingMode, definimos que o
mapa irá
    //rastrear a localização do usuário, centralizando onde
ele estiver
    mapView.userTrackingMode = .follow

    mapView.delegate = self

    requestAuthorization()
}
```

Código-fonte 9 – Definindo *delegate* do *Map View*
Fonte: Elaborado pelo autor (2019)

Vamos implementar o método *showRoute*. Nele, nós iremos realizar a pesquisa da rota de maneira semelhante ao que fizemos com a pesquisa dos pontos de interesse, porém utilizando outras classes.

A classe que configura a requisição é a ***MKDirectionsRequest***. Nela, precisamos definir a origem (***source***) e o destino (***destination***), e esses objetos são do tipo ***MKMapItem***. Para construirmos um *MKMapItem*, precisamos utilizar um objeto do tipo ***MKPlacemark***, que é constituído das coordenadas da origem ou destino.

A rota é calculada pela classe ***MKDirections*** e seu método *calculate*, que possui os parâmetros de erro e de resposta do cálculo solicitado. Essa resposta contém um *array* de rotas, que é a propriedade ***routes***. Pegamos a primeira rota deste *array* e recuperamos suas informações. Uma das informações é a propriedade ***polyline***, que contém uma linha vetorial que define o caminho da rota. Essa linha será adicionada ao mapa, de modo que possamos visualizar a rota.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Implemente o método *showRoute* conforme o código-fonte “Implementando método que mostra a rota” e atente para as explicações contidas no código.

```
func showRoute(to destination: CLLocationCoordinate2D) {
    //Objeto de requisição de rota
    let request = MKDirections.Request()

    //Configurando origem e destino. Note que estes objetos
    //são do tipo MKMapItem, e para criar um MKMapItem
    //precisamos criar um MKPlacemark, que pode ser criado a
    //partir de uma coordenada.
    request.source = MKMapItem(placemark:
MKPlacemark(coordinate: mapView.userLocation.coordinate))
    request.destination = MKMapItem(placemark:
MKPlacemark(coordinate: destination))

    //O objeto que faz o cálculo da rota é o MKDirections
    let directions = MKDirections(request: request)
    directions.calculate { (response, error) in
        if error == nil {

            //Abaixo, desembrulhamos o response e também
            //recuperamos a primeira rota da lista de rotas
            //presentes no response
            guard let response = response, let route =
response.routes.first else {return}

            //É possível recuperar uma série de informações
            //sobre a rota, como nome, distância, tempo, etc
            print("Nome:", route.name, "- distância:",
route.distance, "- duração:", route.expectedTravelTime)

            //Além disso, é possível recuperar todos os
            //passos da rota
            for step in route.steps {
                print("Em", step.distance, "metros, ",
step.instructions)
            }

            //Antes de adicionar o overlay, precisamos
            //remover qualquer outra rota apresentada
          self.mapView.removeOverlays(self.mapView.overlays)

            //Agora, vamos adicionar a rota no mapa. Isso é
            //feito utilizando overlay.
            //A propriedade polyline nos dá a linha poligonal
            //que representa a rota, e é esta linha que
            //iremos colocar por cima do mapa.
            //Vamos colocar a rota por cima das ruas e abaixo
            //dos nomes das ruas
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                self.mapView.addOverlays([route.polyline], level:
.aboveRoads)
            }
        }
}
```

Código-fonte 10 – Implementando método que mostra a rota
Fonte: Elaborado pelo autor (2019)

Se você executar o aplicativo agora, verá que não aparece nenhuma rota no mapa. Isso acontece pois os *overlays* do mapa precisam ser renderizados, ou seja, precisam que cores, tamanho etc. sejam atribuídos para que sejam visíveis. No entanto, caso clique em Pin que está representando um cinema e depois clicar no botão "Traçar rota", você verá a rota no console.

O método que faz isso se encontra no protocolo que define o delegate do *Map View*, e é o método ***mapView(_ mapView: MKMapView, rendererFor overlay: MKOverlay) -> MKOverlayRenderer***. Por meio dele, criamos um objeto do tipo MKOverLayRenderer, que é um renderizador de *overlays*.

Nesse método, vamos criar um renderizador específico para *overlays* do tipo *polyline*, e definiremos a cor e a espessura da nossa rota.

Implemente o código-fonte "Implementando *renderer* para a rota" **dentro da extensão que criamos** para implementar o protocolo ***MKMapViewDelegate***.

```
func mapView(_ mapView: MKMapView, rendererFor overlay:
MKOverlay) -> MKOverlayRenderer {
    //Se o overlay for uma MKPolyline, usamos o renderer
correspondente
    if overlay is MKPolyline {
        //Criando um renderer para polylines
        let renderer = MKPolylineRenderer(overlay: overlay)

        //Definindo sua espessura e cor
        renderer.lineWidth = 8.0
        renderer.strokeColor = .blue

        return renderer
    } else {
        return MKOverlayRenderer(overlay: overlay)
    }
}
```

Código-fonte 11 – Implementando *renderer* para a rota
Fonte: Elaborado pelo autor (2019)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Rode o aplicativo, acesse o mapa, pesquise por algum ponto de interesse e toque em alguma das *annotations* adicionadas. Nesse momento, no alerta que surgir, toque em **Traçar rota**. Você irá notar que uma linha azul vai se formar de onde você estiver até o ponto de interesse selecionado.

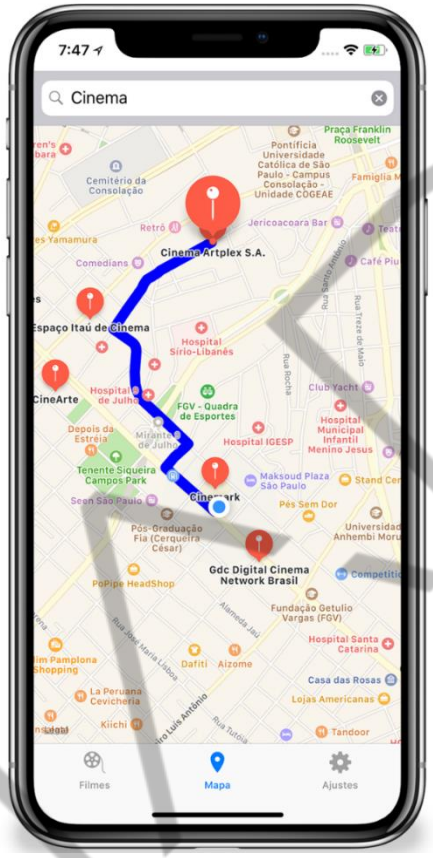

Figura 9 – Rota traçada
Fonte: Elaborado pelo autor (2019)

2.2 Usando *WebView* em seus Apps

Em certas ocasiões é necessário mostrar informações de sites dentro de um aplicativo, e em iOS temos um componente que serve justamente para esse propósito, o *Web View* (***WKWebView***). Por meio dele, podemos apresentar um site para o usuário, permitindo que ele navegue sem a necessidade de sair do app.

Adicione uma nova *View Controller* no *storyboard* e posicione-a ao lado da cena do mapa. Coloque um botão no topo da cena, defina seu texto como **Fechar** e seu *frame* como X: **16**, Y: **59**, *Width*: **90** e *Height*: **35**. Adicione *contraints* para o topo e o lado esquerdo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos agora adicionar o componente *WebKit View*. Posicione-o de modo que seu frame fique X: **0**, Y: **93**, *Width*: **393** e *Height*: **759** e adicione as *constraints* de topo (34), esquerda, direita e base (todas com 0).

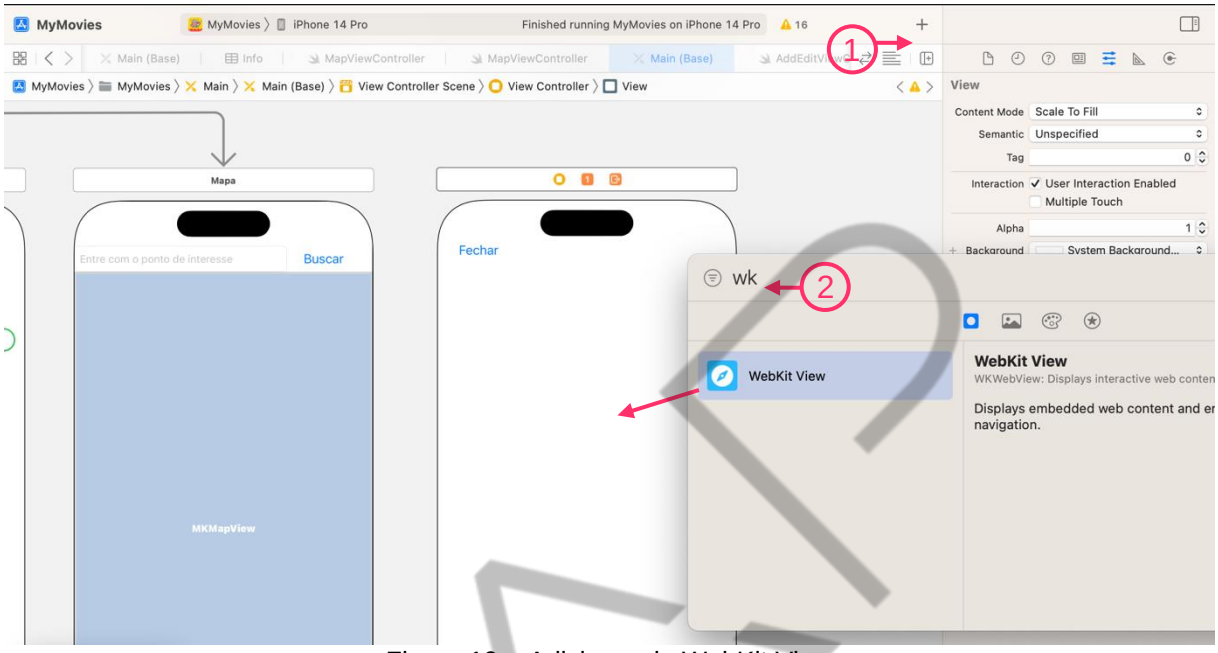

Figura 10 – Adicionando WebKit View
Fonte: Elaborado pelo autor (2023)

Crie uma classe do tipo *UIViewController* chamada ***SiteViewController*** e vincule-a a essa cena pelo Identity Inspector. Agora, crie o ***IBOutlet*** para a *Web View* com o nome ***webView*** e uma ***IBAction*** para o botão *Fechar* chamada **close**. Nessa classe, crie uma variável chamada **site** do tipo *String* com conteúdo vazio **e importe o Framework WebKit** para que possamos ter acesso aos recursos da WKWebView.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

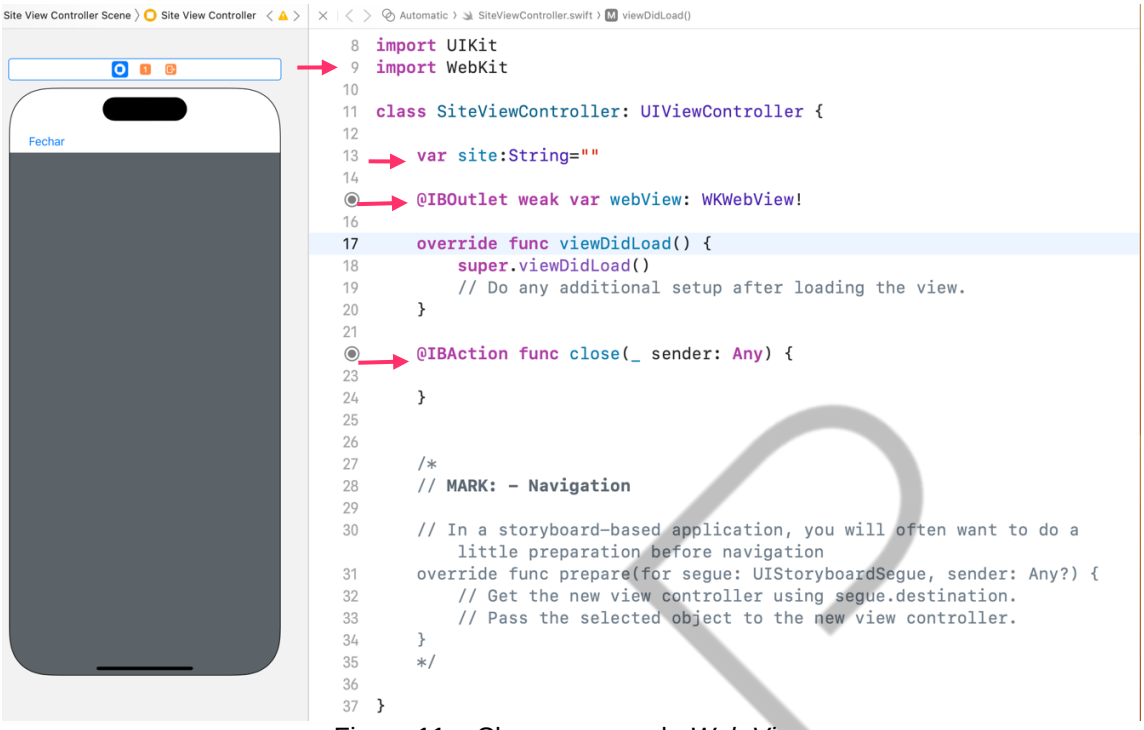

Figura 11 – Classe e cena da *Web View*
Fonte: Elaborado pelo autor (2023)

Precisamos passar o código da tela de mapas para a nossa tela de *Web View*. Como a tela de *Web View* será chamada pela tela do mapa, crie uma segue (*show*) da cena do mapa até a cena da *Web View*. Selecione a segue e defina seu *Identifier* como **webSegue**.

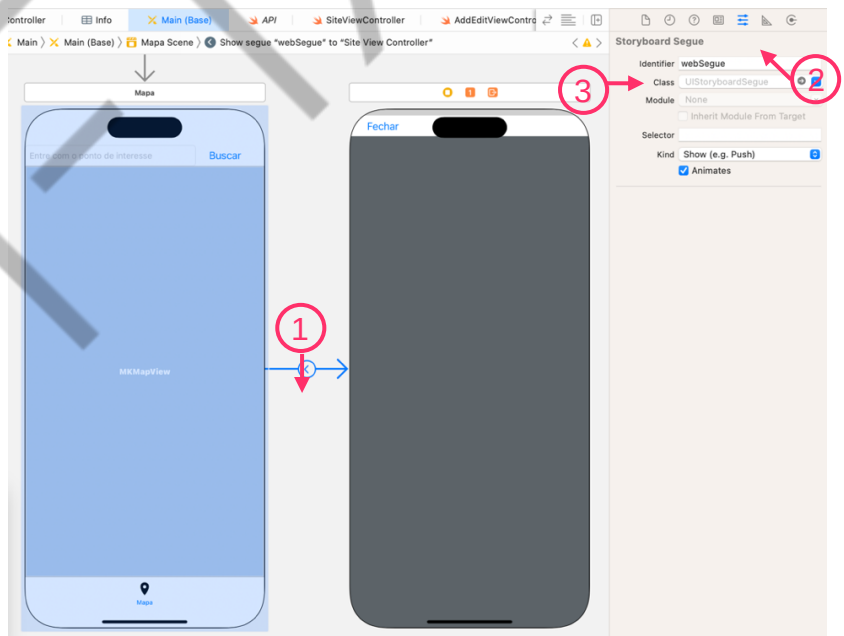

Figura 12 – Criando segue *webSegue*
Fonte: Elaborado pelo autor (2023)

Agora, acesse a classe *MapViewController* e vá até o método *mapView(_ mapView: MKMapView, didSelect view: MKAnnotationView*). Nele, no momento em

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

que o usuário tocar no botão **Acessar site**, vamos chamar a segue (trecho em negrito) passando a url do site. Modifique esse método, de modo que o trecho de código no qual criamos a *Action* de acessar site fique igual ao código-fonte "Chamando segue *webSegue*".

```
if let subtitle = annotation.subtitle, let url = subtitle {
    let urlAction = UIAlertAction(title: "Acessar site",
style: .default) { (action) in
        self.performSegue(withIdentifier: "webSegue", sender:
url)
    }
    alert.addAction(urlAction)
}
```

Código-fonte 12 – Chamando segue *webSegue*
Fonte: Elaborado pelo autor (2023)

Para que o endereço do site seja repassado à tela da *Web View*, implemente o método *prepare* (for segue: *UIStoryboardSegue, sender: Any*?) conforme código-fonte "Enviando URL para nova tela".

```
override func prepare(for segue: UIStoryboardSegue, sender:
Any?) {
    if let vc = segue.destination as? SiteViewController, let
sender = sender as? String {
        vc.site = sender
    }
}
```

Código-fonte 13 – Enviando URL para nova tela
Fonte: Elaborado pelo autor (2019)

Vamos implementar o código para carregar esse site na *Web View*. Antes, precisamos implementar, no info.plist, uma entrada que permita que o app acesse sites não seguros. Por padrão, o iOS não permite que apps acessem sites http, apenas https, mas isso pode ser contornado utilizando a entrada ***App Transport Security Settings***, juntamente com ***Allow Arbitrary Loads***, definindo esse último como ***YES***. Implemente isso no info.plist de modo que ele fique como na figura **"Permitindo o uso de sites não-seguros".** Perceba que *Allow Arbitraty Loads* é uma subentrada de *App Transport Security Settings*, portanto você vai precisar, depois que informar o ítem "*App Transprot Security Settings*", clicar na seta de forma que a ala aponte para baixo indicando que a próxima entrada (+) será uma subentrada de "*App Transprot Security Settings*".

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

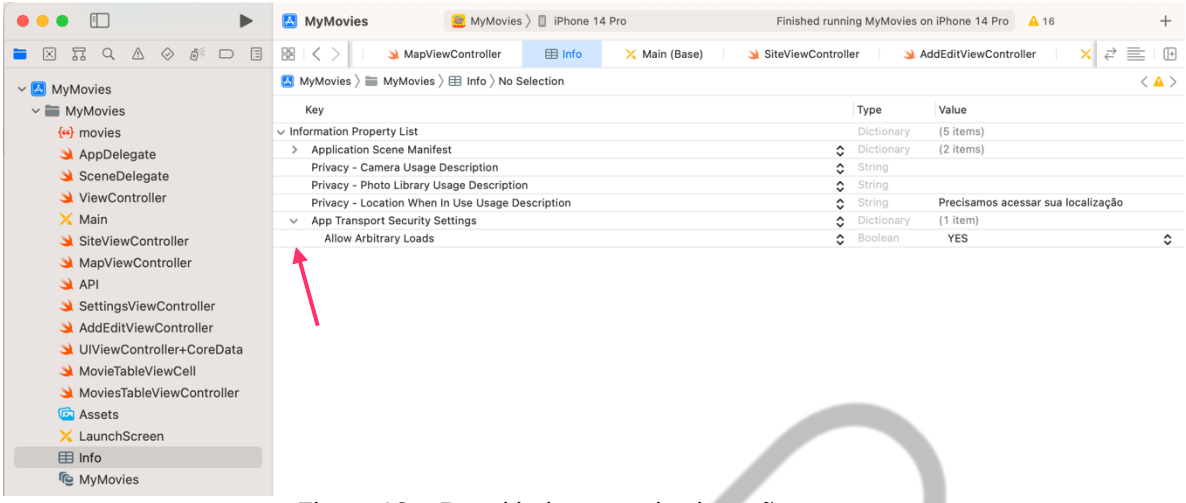

Figura 13 – Permitindo o uso de sites não seguros
Fonte: Elaborado pelo autor (2019)

Abra a classe *SiteViewController* e implemente o código-fonte "Classe *SiteViewController*". Aqui, estamos criando um objeto do tipo URL contendo o endereço web do site selecionado e, a partir dele, criamos uma requisição web (***URLRequest***). De posse desse objeto, solicitamos que a web view faça o carregamento do site por meio do método ***load(_ request: URLRequest)***. Também implementamos a ação de fechar.

No final, a classe *SiteViewController* ficará como mostra o código-fonte "Classe *SiteViewController*".

```
import UIKit
import WebKit

class SiteViewController: UIViewController {

    @IBOutlet weak var webView: WKWebView!
    var site: String = ""

    override func viewDidLoad() {
        super.viewDidLoad()
        if let url = URL(string: site) {
            let request = URLRequest(url: url)
            webView.load(request)
        }
    }

    @IBAction func close(_ sender: UIButton) {
        dismiss(animated: true, completion: nil)
    }
}
```

Código-fonte 14 – Classe *SiteViewController*
Fonte: Elaborado pelo autor (2019)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.3 Vídeos: AVPlayer, AVPlayerViewController

Nosso aplicativo está quase pronto. Uma das partes que ficaram faltando foi tocar o trailer dos filmes, e, para isso, faremos uso de duas classes que, em conjunto, nos fornecem as ferramentas necessárias para a visualização de arquivos de vídeo. São elas: ***AVPlayer*** e ***AVPlayerViewController***. Acessamos essas classes por meio do framework ***AVKit***.

AVPlayer é uma classe utilizada para playback de arquivos de áudio e vídeo, sejam locais ou oferecidos via *streaming* HTTP. Ela é capaz de tocar apenas um arquivo de mídia por vez.

O *AVPlayer* é um objeto não-visual, ou seja, ele é incapaz de apresentar o vídeo que está manipulando. Para isso, utilizamos a classe *AVPlayerViewController*. Ela é responsável por apresentar visualmente o vídeo, bem como todos os controles para o seu *playback* e demais recursos de mídia, fornecendo uma experiência completa de visualização de vídeo.

Acesse o *storyboard*, selecione a cena de visualização de filme e por consequência a classe ViewController, crie um *IBOutlet* para o botão play com o nome ***btPlay***. Crie também uma *IBAction* para esse botão, com o nome ***play***.

Acesse a classe *ViewController* e crie duas propriedades: uma para representar o objeto *AVPlayer* e outra para o *AVPlayerViewController*. Crie, abaixo da propriedade trailer, conforme o código-fonte "Objetos *AVPlayer* e *AVPlayerViewController*".

```
var trailer: String = ""

//Criando objetos AVPlayer e AVPlayerViewController
var moviePlayer: AVPlayer?
var moviePlayerController: AVPlayerViewController?
```

Código-fonte 15 – Objetos *AVPlayer* e *AVPlayerViewController*
Fonte: Elaborado pelo autor (2019)

As duas novas linhas irão disparar um erro. Sendo assim, vá no topo da classe e inclua o **import AVKit**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos agora criar um método para instanciar esses objetos e prepará-los para tocar o trailer do filme. Crie o método *prepareVideo*() com o código "Preparando *moviePlayer* e *moviePlayerViewController"*. Esse código instancia o *moviePlayer*, passando para ele a URL criada a partir da *string,* contendo o endereço do trailer, e depois define que o *moviePlayer* será o player utilizado pelo *AVPlayerViewController*.

```
func prepareVideo() {
    //Criando a URL com o endereço do trailer do filme
    guard let url = URL(string: trailer) else {return}

    //Instanciando o moviePlayer e moviePlayerController
    moviePlayer = AVPlayer(url: url)
    moviePlayerController = AVPlayerViewController()

    //Definimos que o moviePlayer será o player de vídeo da
    //controller
    moviePlayerController?.player = moviePlayer

    //Definindo que os controles de playback do vídeo serão
    //mostrados
    moviePlayerController?.showsPlaybackControls = true
}
```

Código-fonte 16 – Preparando *moviePlayer* e *moviePlayerViewController*
Fonte: Elaborado pelo autor (2019)

Vamos chamar esse método assim que o endereço do trailer do filme for recuperado, ou seja, faremos a chamada dentro do método *loadTrailers*, logo após termos alimentado a variável trailer. No final, o método *loadTrailers* ficará como mostra o código-fonte "Chamando *prepareVideo"* (inclusão da linha no trecho em negrito).

```
func loadTrailers(title: String) {

    //Chamamos o método loadTrailiers da classe API e na
closure
    //vamos recuperar o primeiro resultado da lista de
resultados.
    API.loadTrailers(title: title) { (apiResult) in
        guard let results = apiResult?.results, let trailer =
results.first else {return}

        //Aqui, pegamos a url do trailer e atribuímos à nossa
variável trailer.
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        //Estamos fazendo isto na thread main.
        DispatchQueue.main.async {
            self.trailer = trailer.previewUrl
            self.prepareVideo()
        }
    }
}
```

Código-fonte 17 – Chamando *prepareVideo*
Fonte: Elaborado pelo autor (2019)

Só nos falta implementar na action do botão play. Para mostrar o vídeo, basta apresentarmos a *moviePlayerViewController* por meio do método *present* e, na *closure* de *completion* desse método, chamar o método play do *moviePlayer* para que o trailer comece a tocar.

O código da *action* ficará como apresentado no código-fonte "Apresentando e tocando o trailer".

```
@IBAction func play(_ sender: UIButton) {
    guard let moviePlayerController = moviePlayerController
else {return}
    present(moviePlayerController, animated: true) {
        self.moviePlayer?.play()
    }
}
```

Código-fonte 18 – Apresentando e tocando o trailer
Fonte: Elaborado pelo autor (2019)

Rode o app, cadastre algum filme e, na tela de visualização, experimente tocar o trailer do filme apertando o botão *play*.

Parabéns, você concluiu o desenvolvimento do seu aplicativo! Vamos aprender como publicá-lo na App Store?

2.4 Publicando apps na App Store

Agora que você tem um aplicativo 100% funcional, só resta publicá-lo na *App Store*, a loja de aplicativos da *Apple*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para que você possa publicar seu app, é necessário que você possua uma conta paga de desenvolvedor. A conta que você criou no início do curso é uma conta gratuita e com ela não é possível submeter aplicativos para a loja da Apple.

Para pagar a conta de desenvolvedor, você deverá acessar o site de desenvolvedor da Apple (https://developer.apple.com/) e clicar em ***Account***. Logue com a sua conta e, na tela que aparecer, selecione a opção ***Join the Apple Developer Program***.

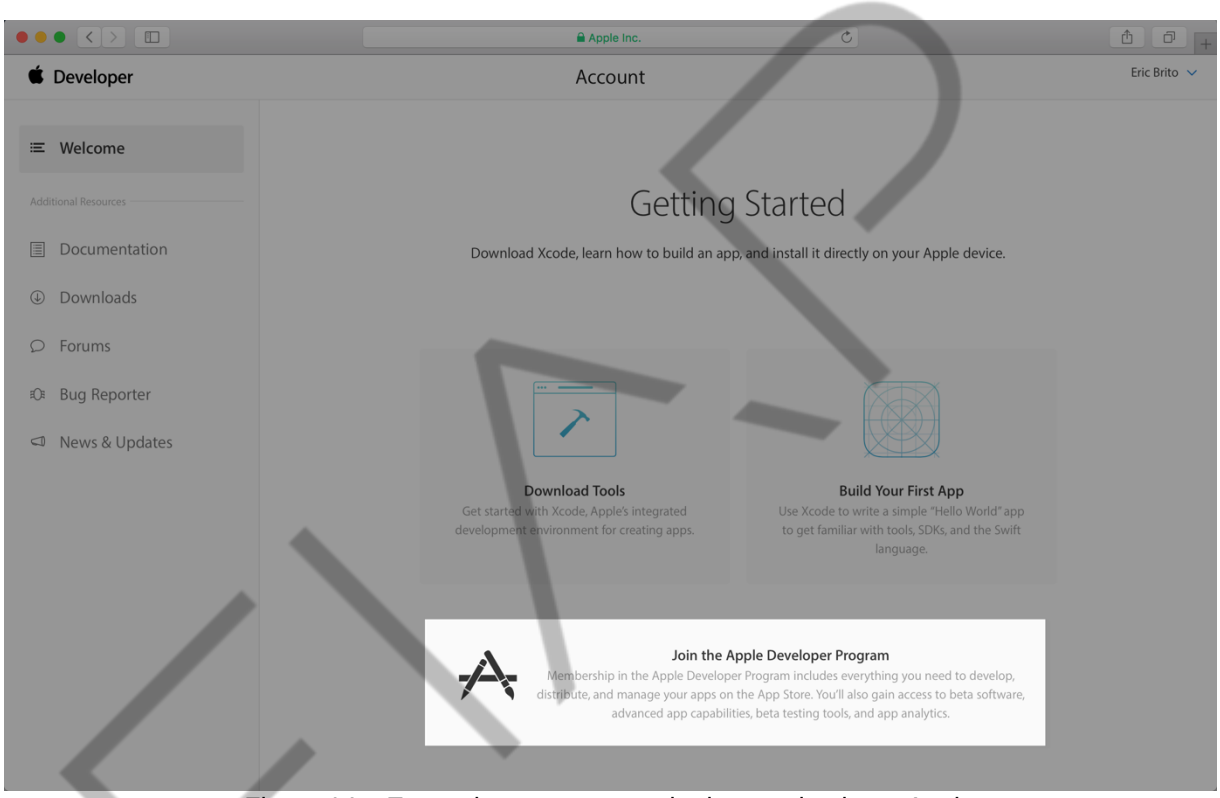

Figura 14 – Entrando no programa de desenvolvedores Apple
Fonte: Elaborado pelo autor (2019)

Na tela que se segue, clique no botão ***Enroll*** localizado no canto superior esquerdo. Na nova tela, role até o final e clique em ***Start Your Enrollment***. Como estamos falando de uma conta para pessoa física, escolha ***Individual / Sole Proprietor / Single Person Business*** na pergunta ***Entity Type*** que aparece no final da tela. Clique em ***Continue*** e, a partir desse momento, você deverá preencher todos os campos e informações necessárias para criar sua conta paga, como nome, telefone, endereço, etc. Ao final, clique em ***Purchase*** para concluir o processo. Será cobrado o valor de 99 dólares do cartão de crédito vinculado à sua conta.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Depois disso, certifique-se de ter associado seu projeto à sua conta. Caso não o tenha feito, abra o projeto, clique no nome e na tela que se abre, na aba ***General*** vá à seção *Signing* e associe a sua conta ao projeto. Caso você ainda não tenha configurado a sua conta no *Xcode*, irá aparecer o botão ***Add Account....*** Ele o levará ao painel de configuração da conta, onde você pode cadastrá-la.

Após fazer isso, e se a opção ***Automatically manage signing*** estiver marcada, o *Xcode* irá criar e assinar um certificado de desenvolvimento para você automaticamente. Esse certificado é necessário para garantir que a sua conta esteja vinculada ao seu *Mac* e, dessa forma, esteja apta a desenvolver o aplicativo que posteriormente será publicado na loja. Repare que, após associar a sua conta, é apresentada, durante um curto período, a mensagem de que este certificado está sendo gerado. Também é possível gerar o certificado por meio do portal do desenvolvedor, porém, da forma descrita acima, o *Xcode* faz isso para você automaticamente.

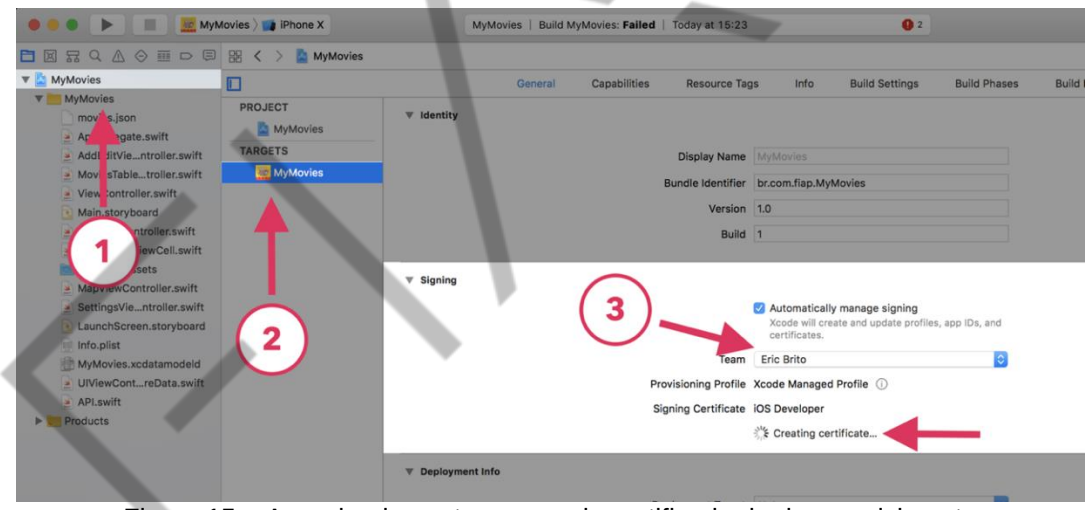

Figura 15 – Associando conta e gerando certificado de desenvolvimento
Fonte: Elaborado pelo autor (2019)

Você pode confirmar que um certificado de desenvolvimento foi gerado logando no portal de desenvolvedor (https://developer.apple.com/account/) e acessando a área *Certificates*, *IDs* & *Profiles*. Uma nova página será aberta e nela você pode verificar seu certificado, que é acessado pelo menu lateral na área ***Certificates -> All***.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

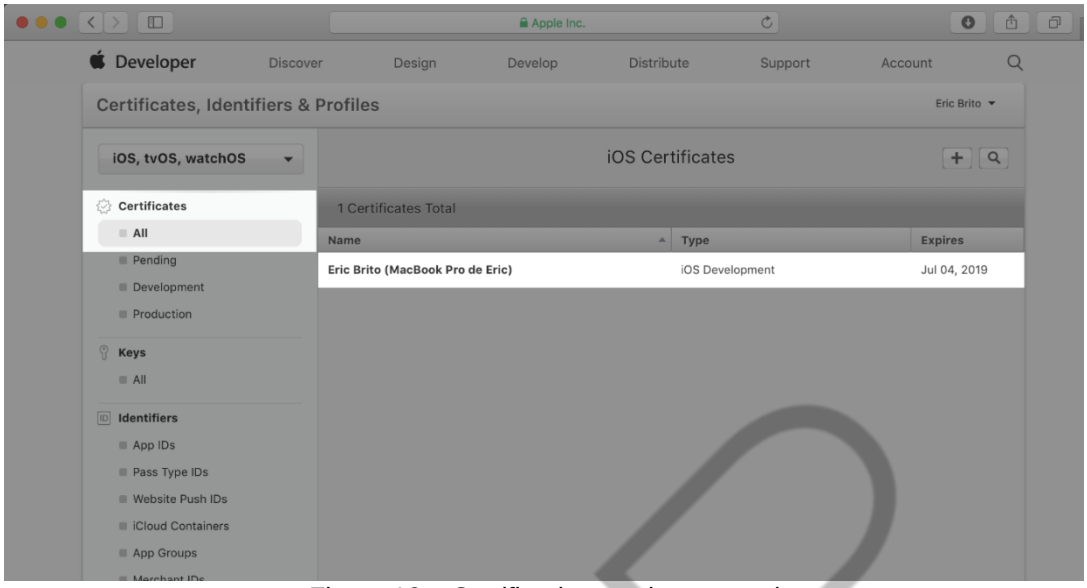

Figura 16 – Certificado gerado no portal
Fonte: Elaborado pelo autor (2019)

Agora, será necessário criar um certificado de produção, ou seja, um certificado que lhe permita submeter seu aplicativo à *App Store*. Nesta mesma tela, clique no botão **+**, que se encontra no canto superior direito (ao lado da lupa) e, na tela que se abre, selecione ***App Store and Ad Hoc*** na seção ***Production***. Vá até o final da página e clique em ***Continue***.

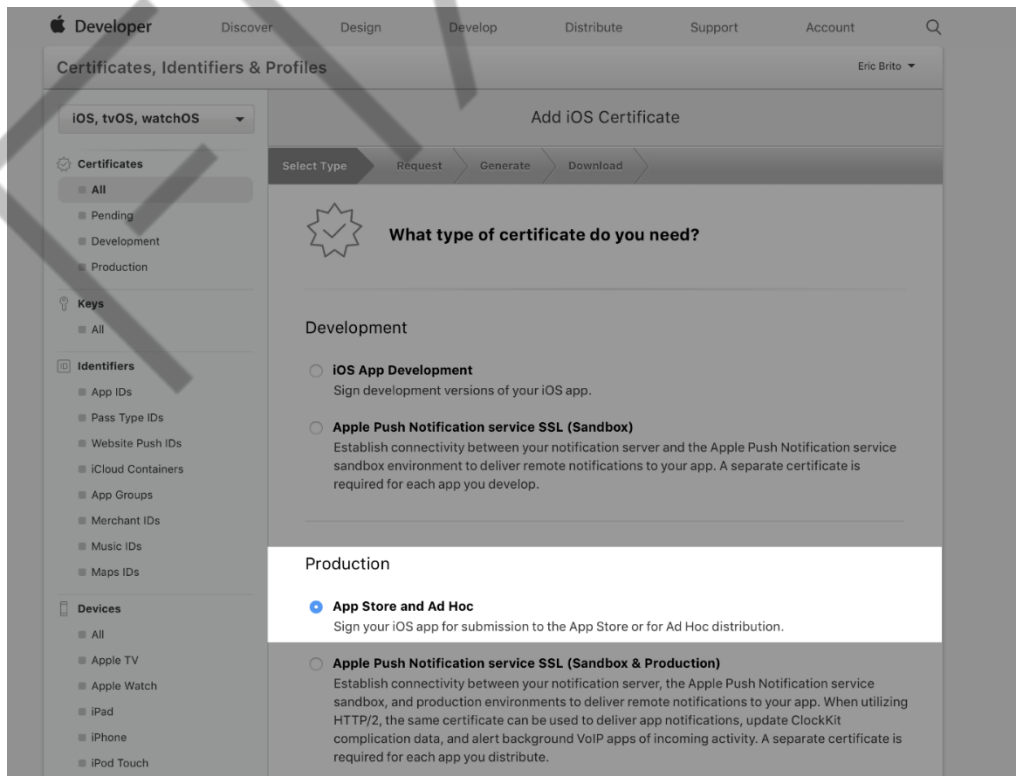

Figura 17 – Iniciando criação de certificado de produção
Fonte: Elaborado pelo autor (2019)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na tela que se segue, encontram-se algumas instruções para a criação do certificado. Você terá que abrir o aplicativo **Acesso às chaves** (aperte *Command* + Barra de espaço e digite Acesso às chaves) e, depois de aberto, você precisa seguir estes passos:

- Clique no menu **Acesso às chaves -> Assistente de Certificado -> Solicitar um Certificado de uma Autoridade de Certificação**.
- Na tela que se abre, preencha o endereço de e-mail da sua conta, seu nome e marque a opção **Salvar no disco**.
- Escolha um local e salve o arquivo.
- Clique em **OK**.

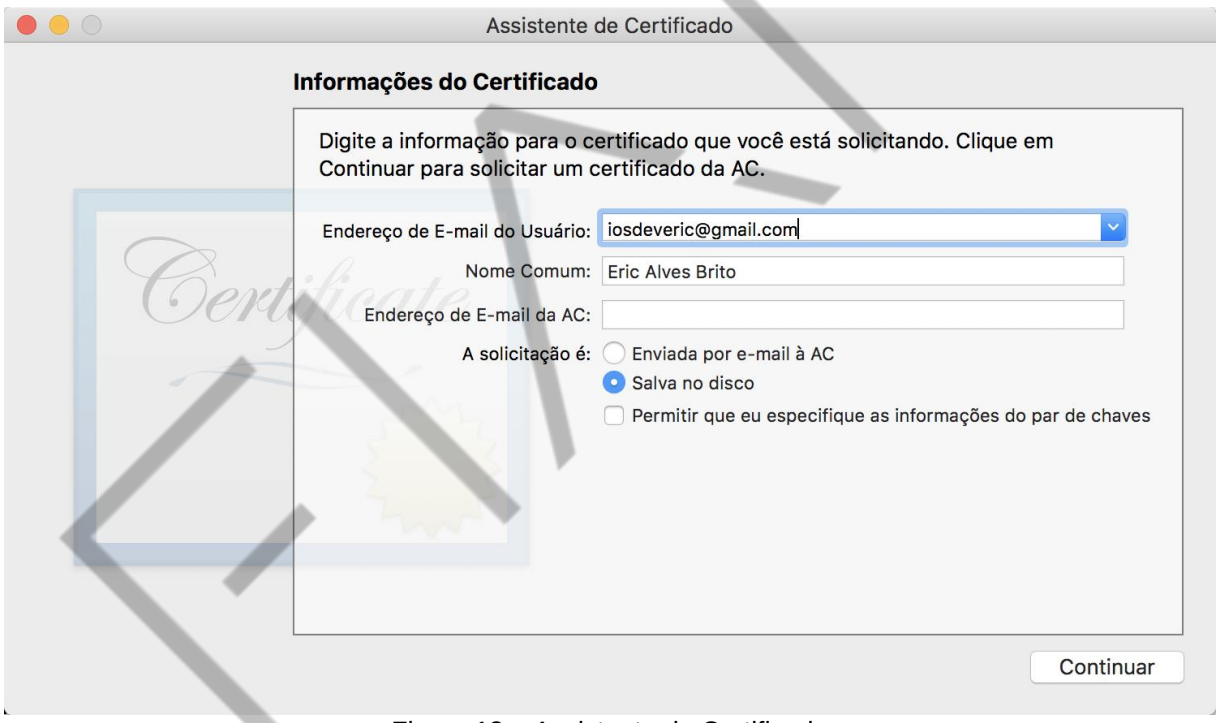

Figura 18 – Assistente de Certificado
Fonte: Elaborado pelo autor (2019)

Agora, volte ao portal de desenvolvedor e clique em Continue. Na próxima tela, clique no botão ***Choose File...*** e escolha o arquivo salvo. Clique em ***Continue***. Depois disso, seu certificado será criado e você pode baixá-lo em seu computador clicando em ***Download***. Depois de salvo, dê dois cliques no arquivo para que ele seja instalado em seu *Mac*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Falta, agora, criar o *App ID* para o seu aplicativo no portal. Essa etapa é necessária, pois será esse *ID* que iremos utilizar para efetivamente cadastrar o aplicativo na *App Store*.

Volte ao portal do desenvolvedor e clique na opção ***App IDs***, que se encontra no menu ***Identifiers***. Clique no botão + (no canto superior direito), e será aberta a tela de Registro de *App ID*.

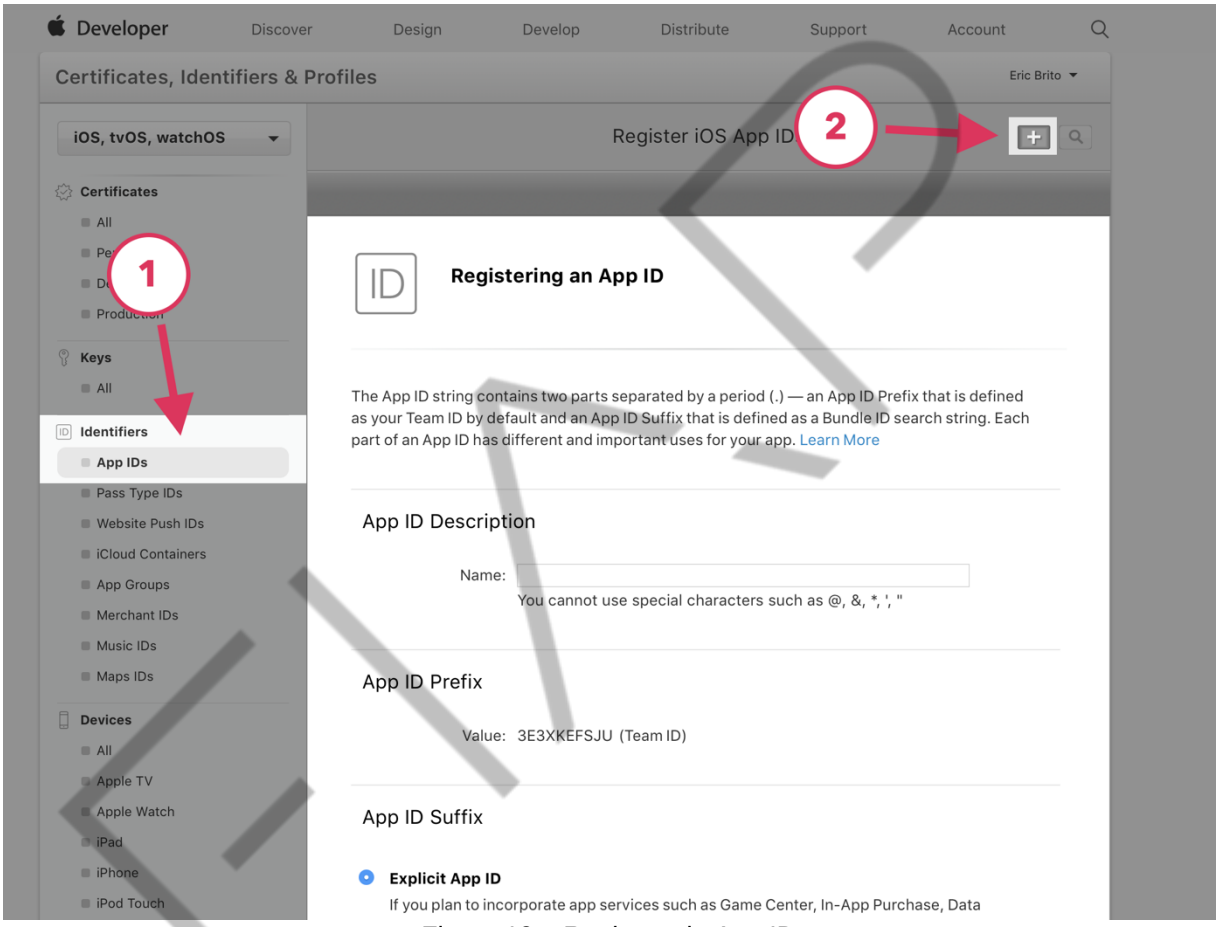

Figura 19 – Registrando App ID
Fonte: Elaborado pelo autor (2019)

Agora, vamos preencher as informações do app. Defina uma descrição para esse *App ID* no campo *Add ID Description* (vamos colocar ***MyMovies***). Mais abaixo, certifique-se de que a opção ***Explicit App ID*** esteja marcada e, no campo, coloque o *bundle identifier* do aplicativo (como não podemos repetir o mesmo *bundle identifier*, sugiro que coloque, por exemplo, br.com.**seunome**.MyMovies, sem se esquecer de também trocar o *bundle identifier* do aplicativo para esse novo lá no Xcode). Siga até o final da página e clique em ***Continue***.

Na tela que segue, clique em ***Register*** e na sequência ***Done***.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

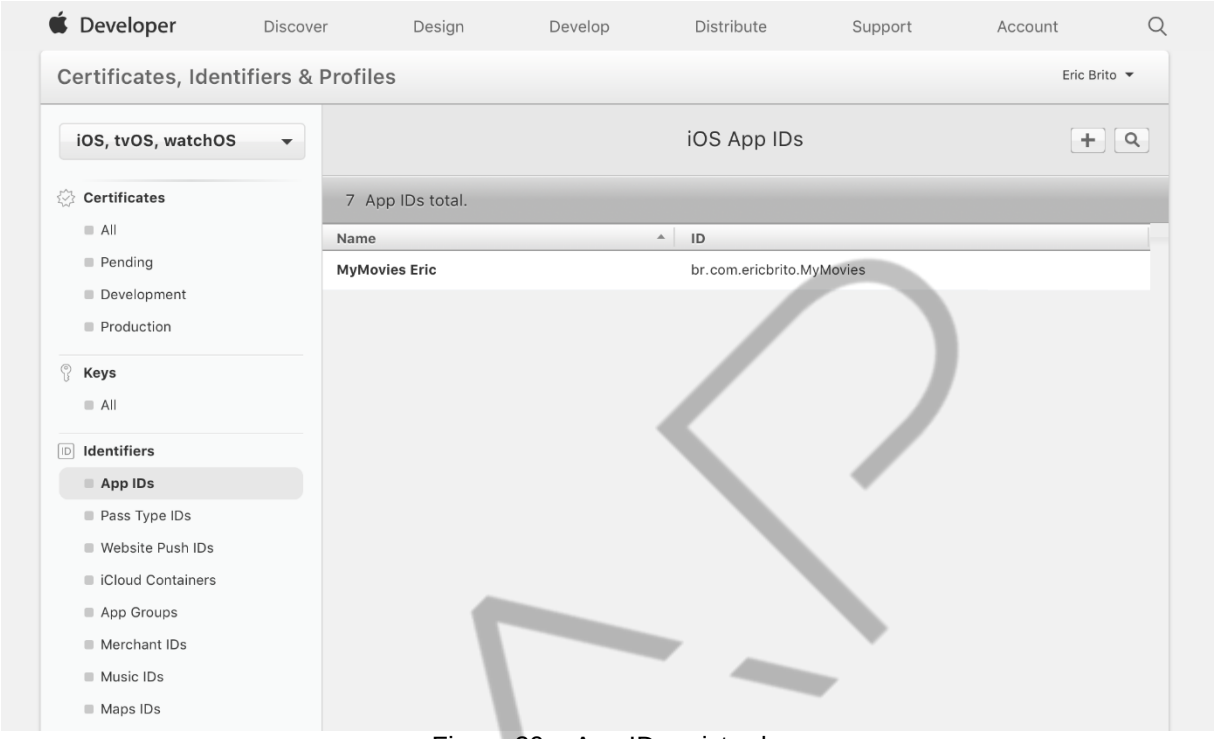

Figura 20 – App ID registrado
Fonte: Elaborado pelo autor (2019)

Agora, precisamos configurar o nosso app na App Store, e isso é feito pelo site *App Store Connect*, que pode ser acessado neste link: (https://appstoreconnect.apple.com). Entre com sua conta de desenvolvedor e, na tela que segue, clique na opção **Meus apps**.

Na próxima tela, clique no botão **+** que se encontra no canto superior esquerdo e escolha **Novo app**. No painel que surgir, você deve preencher todas as informações do aplicativo, como o nome, idioma principal, *ID* do pacote (aqui você deve escolher o *App ID* criado anteriormente) e um SKU, que é um código único que será atribuído ao seu app (qualquer código vale desde que seja único, ou seja, não tenha sido usado por outro App). Ao finalizar, clique em **Criar**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

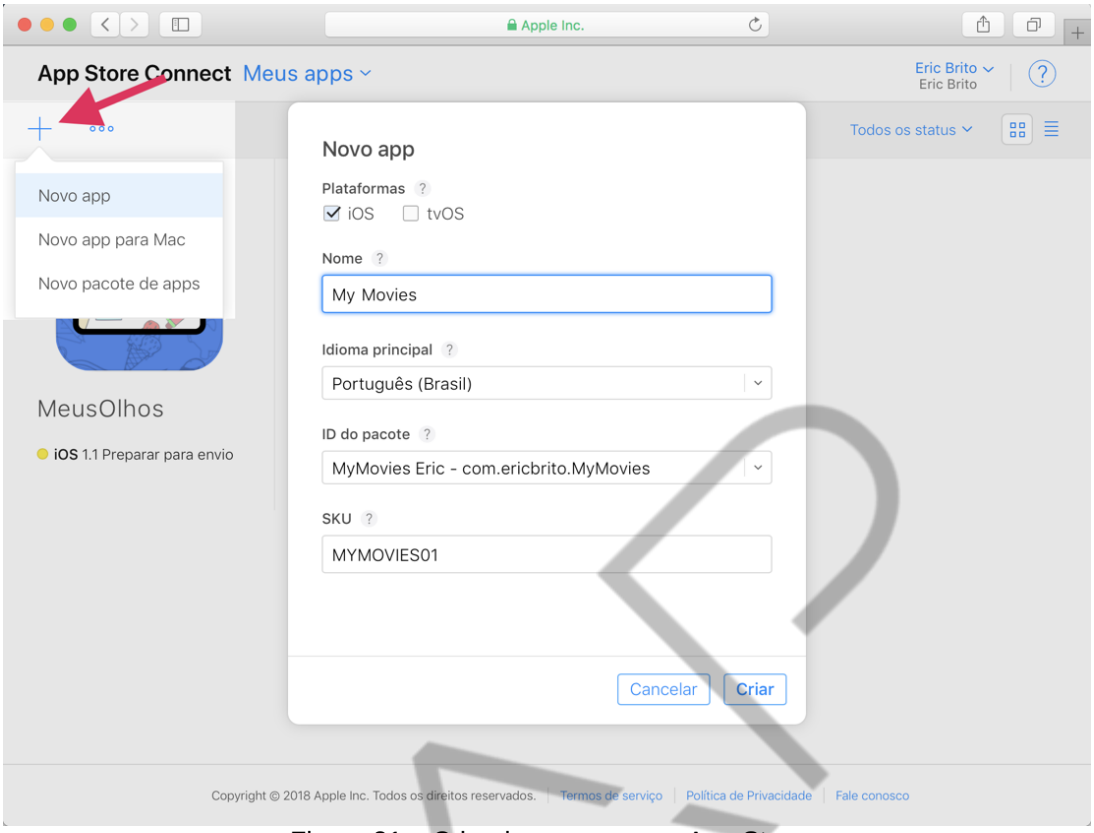

Figura 21 – Criando novo app na App Store
Fonte: Elaborado pelo autor (2019)

Depois de criado, você terá acesso à tela de configuração do seu aplicativo na App Store. No submenu **Informações do app** você pode configurar novamente o **nome** do app, bem como seu **subtítulo** e a **URL da Política de privacidade**. Também é necessário informar a **Categoria primária** do aplicativo (Compras, educação, entretenimento, esportes, etc.). Depois de preenchido, clique em **Salvar**, no canto superior direito.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

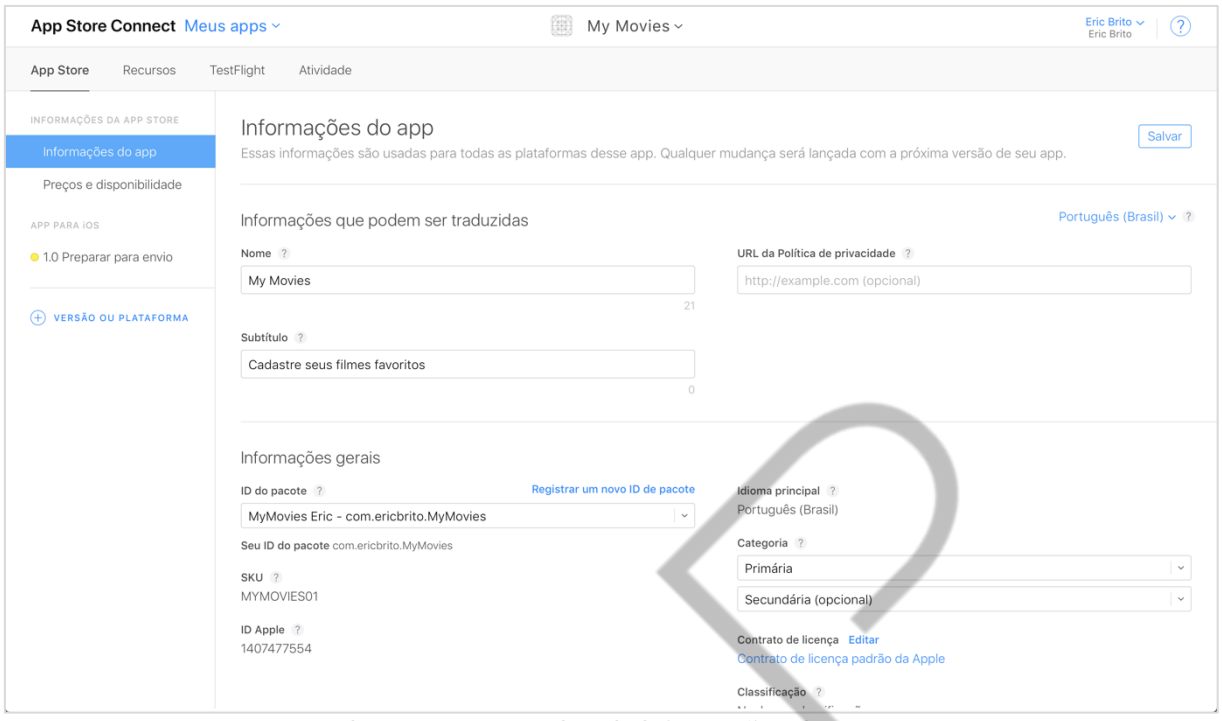

Figura 22 – Preenchendo informações do App
Fonte: Elaborado pelo autor (2019)

Em **Preços e disponibilidade** é possível escolher quanto o aplicativo irá custar, ou defini-lo como gratuito, além de outras informações relativas a descontos, pré-venda etc. Vamos definir nosso app como **Grátis**. Clique em **Salvar**.

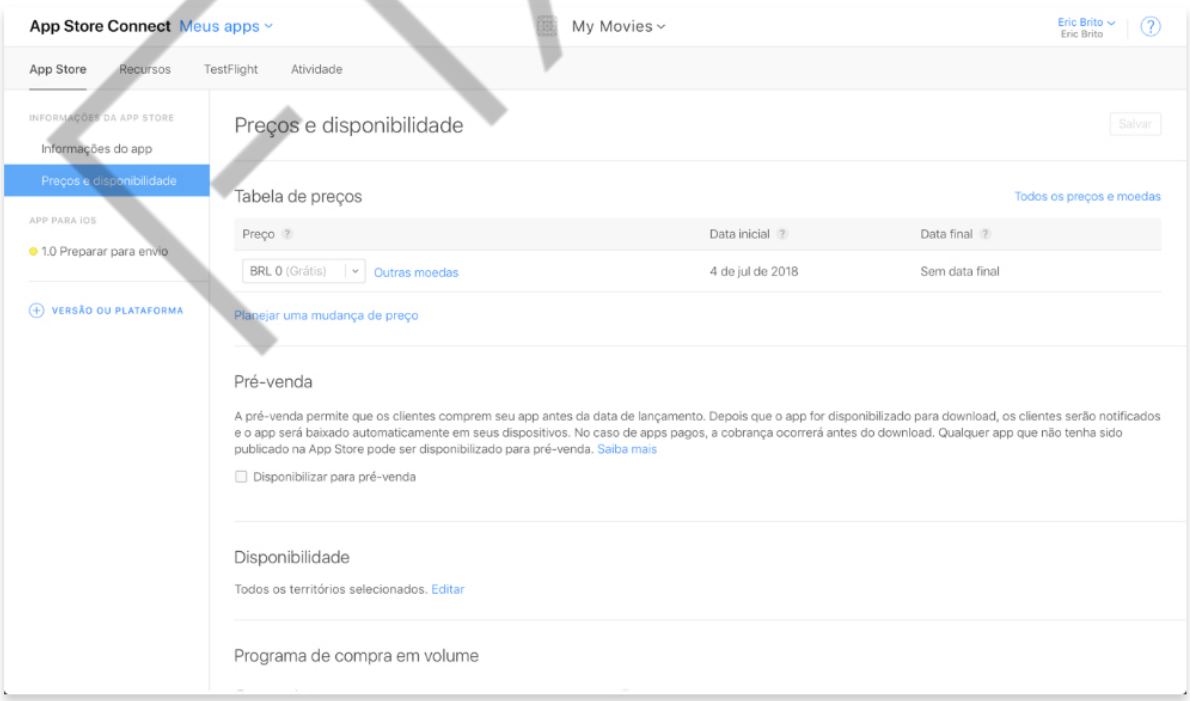

Figura 23 – Preços e disponibilidade
Fonte: Elaborado pelo autor (2019)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora, vamos clicar na opção **1.0 Preparar para envio**. É aqui que iremos configurar as fotos de marketing do aplicativo, ou seja, as imagens que vão aparecer na App Store quando o usuário encontrar o app. Essas imagens precisam estar em tamanhos específicos, e costuma-se usar screenshots de telas do app nesse local. Você pode submeter fotos para os diversos tamanhos de iPhone, e para saber quais tamanhos essas imagens devem ter, acesse https://help.apple.com/app-store-connect/#/devd274dd925.

Outras informações que você deve cadastrar são o **Texto promocional**, **Descrição, Palavras-chave** e **URL de suporte,** ***Copyright*** e **Classificação**. Além disso, também é preciso preencher as informações de contato, que são o seu **Nome**, **Sobrenome**, **Número de telefone** e **E-mail**. Percorra essa página, cadastre todas as informações solicitadas e clique em **Salvar**.

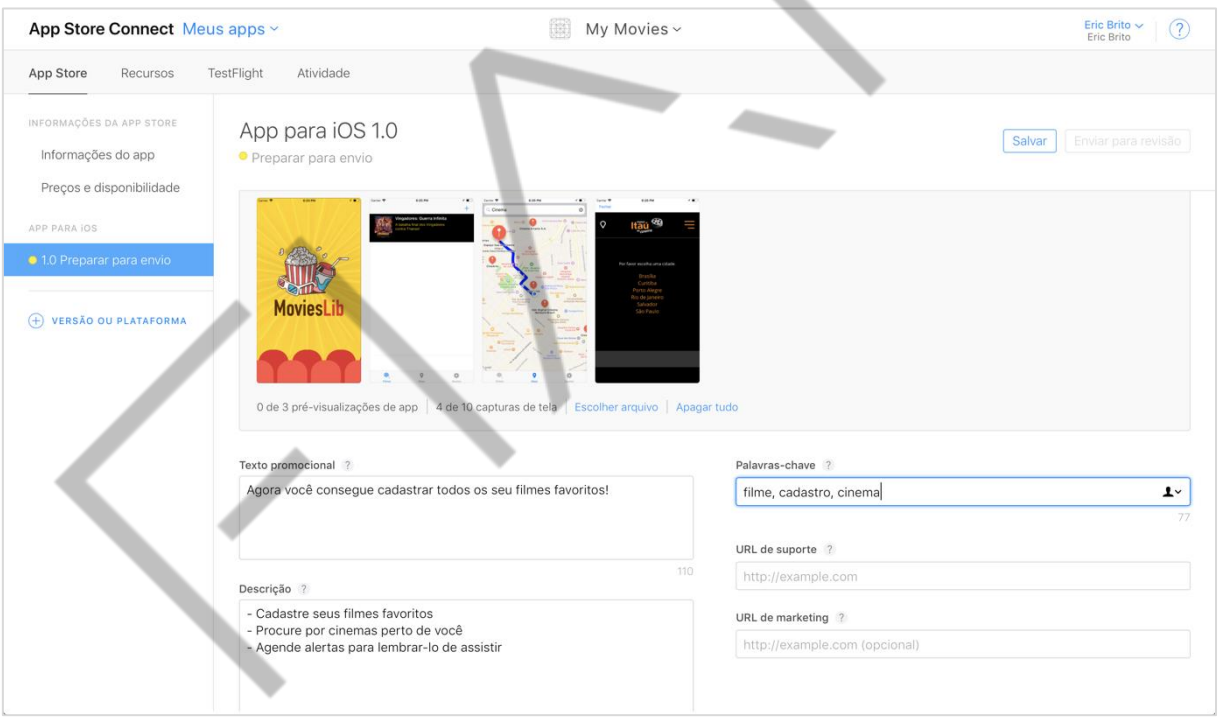

Figura 24 – Preenchendo informações da versão
Fonte: Elaborado pelo autor (2019)

Uma informação fundamental que precisa ser fornecida é o binário, a compilação do arquivo que contém o app em si. Esse arquivo deve ser gerado pelo Xcode e submetido à loja, para que posteriormente você possa selecioná-lo nesta tela.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Volte ao Xcode e selecione, em ***Run Destination,*** (o painel que mostra a lista de simuladores) a opção *Generic iOS Device*. Isso é necessário para gerarmos a compilação do nosso aplicativo.

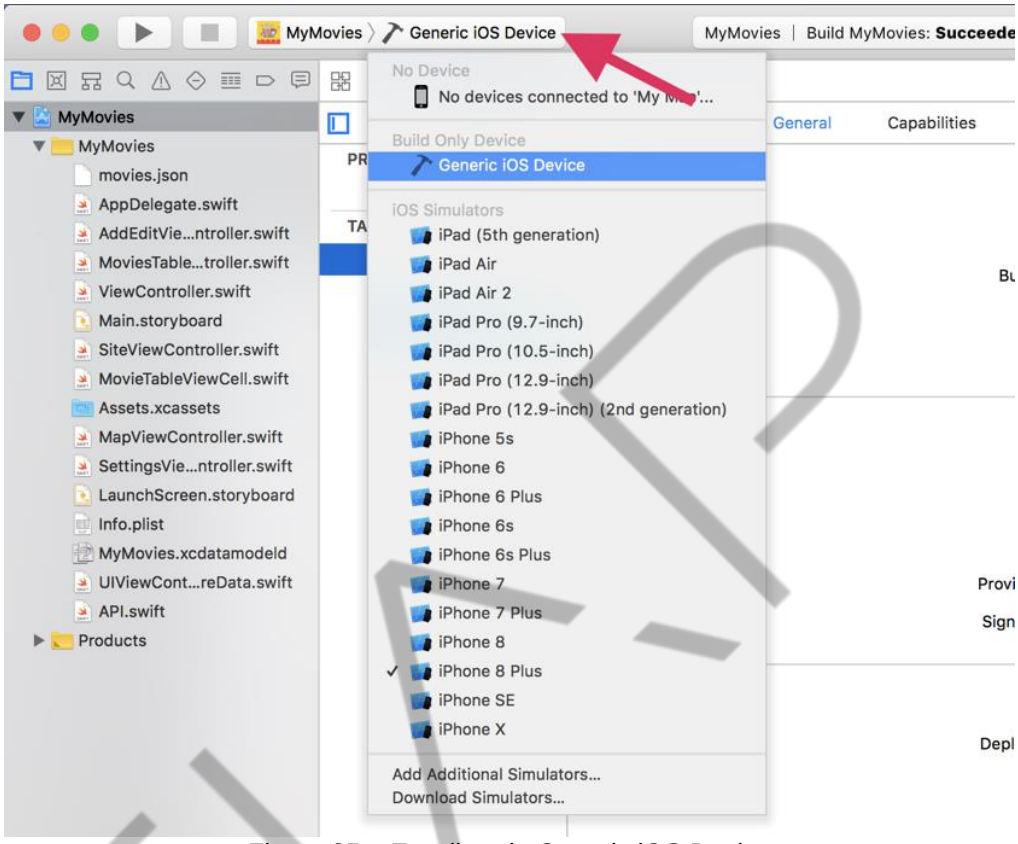

Figura 25 – Escolhendo Generic iOS Device
Fonte: Elaborado pelo autor (2019)

Agora, vá ao menu ***Product*** e clique em ***Archive***. Nesse momento, o processo de compilação começará e, após isso, surgirá a tela ***Archives*** contendo o seu binário. Selecione-o e clique em ***Upload to App Store...***. Clique em *Next* em todas as telas que surgirem e, no final, clique em ***Upload***.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

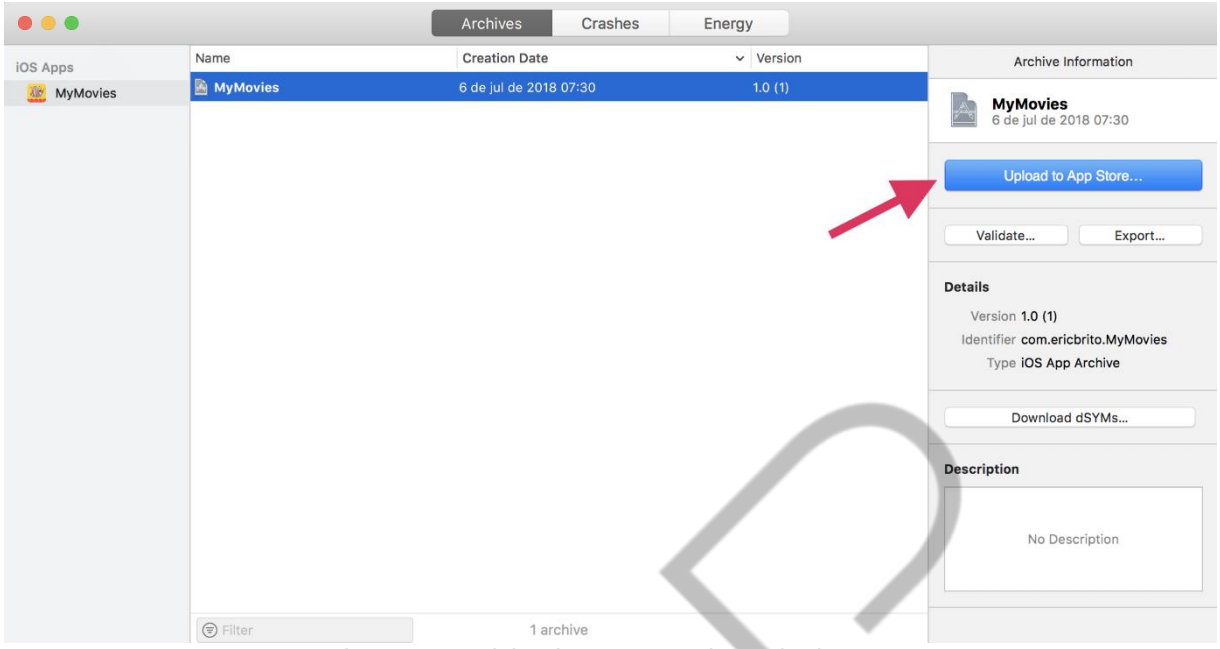

Figura 26 – Iniciando processo de envio do App
Fonte: Elaborado pelo autor (2019)

Esse processo demora um pouco e, ao terminar, caso tudo tenha ocorrido bem, surgirá uma tela com a mensagem ***Upload Successful***. Clique em ***Done***.

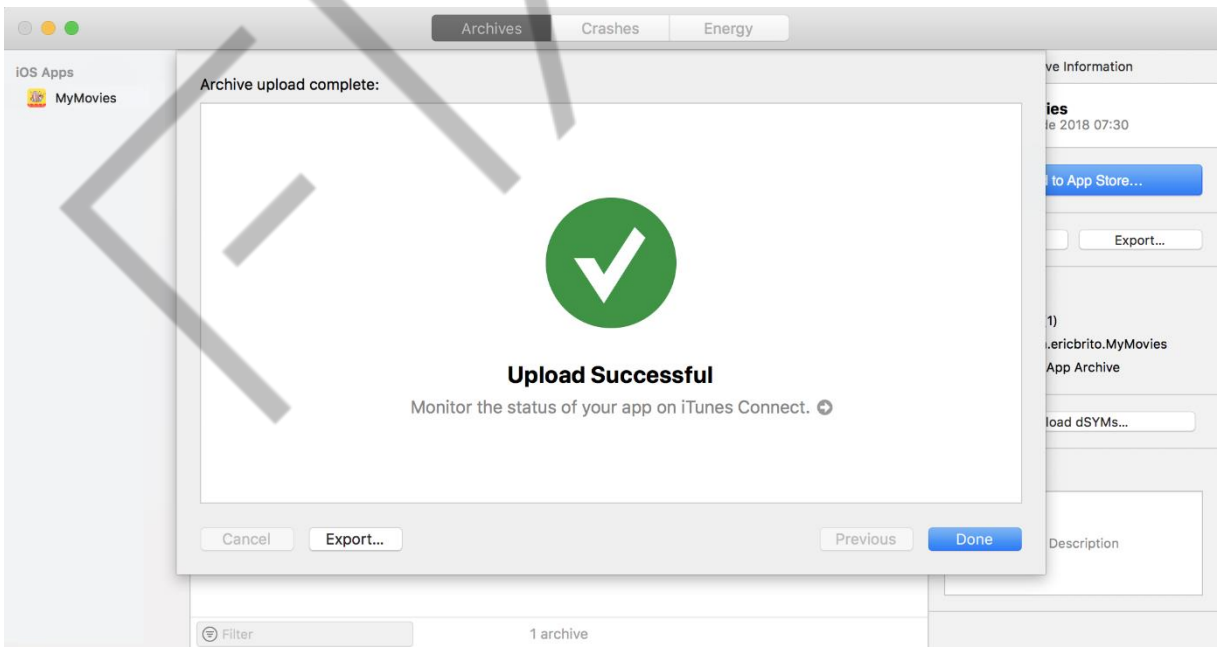

Figura 27 – Upload concluído com sucesso
Fonte: Elaborado pelo autor (2019)

Depois disso, sua compilação (o arquivo que representa o seu aplicativo) terá sido enviada à *App Store* e, na sequência, entrará em processamento pela Apple.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Essa etapa também leva alguns minutos e pode ser acompanhada no site da ***App Store Connect*** na seção **Atividade**, onde você verá que sua compilação está sendo processada.

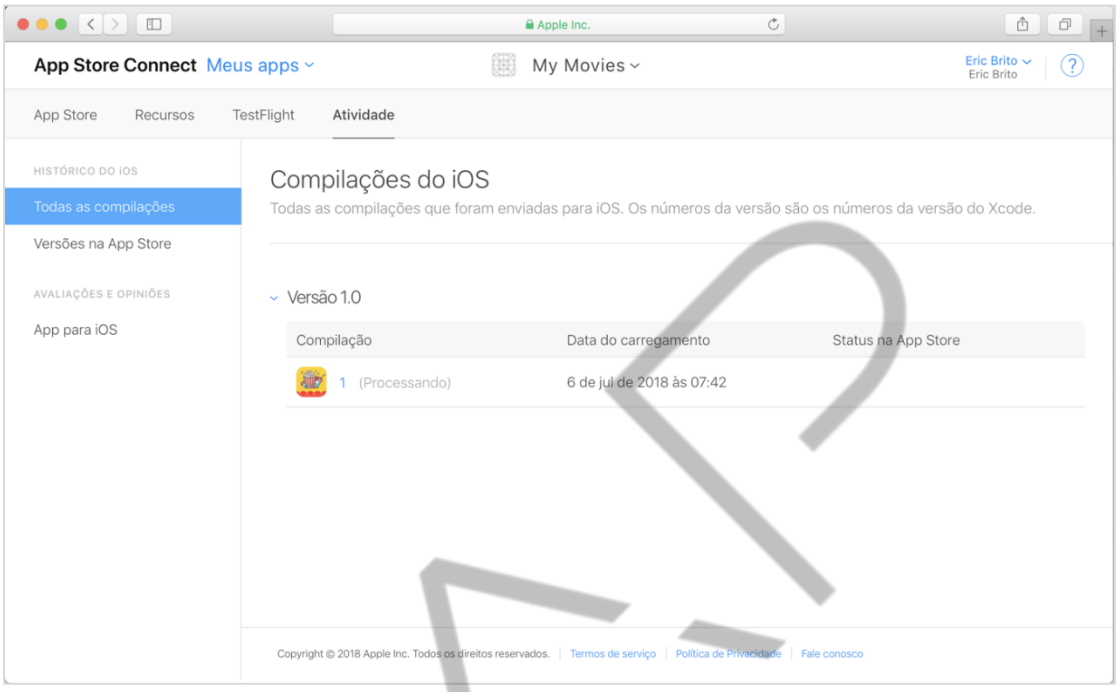

Figura 28 – Atividade
Fonte: Elaborado pelo autor (2019)

Quando o processamento da compilação estiver finalizado, você deverá voltar à aba ***App Store***, na seção **1.0 Preparar para envio**, descendo até a área **Compilação**. Ao lado desse texto terá um botão + que, ao ser clicado, mostra a compilação que você subiu. Selecione-a e clique em **Concluído**.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

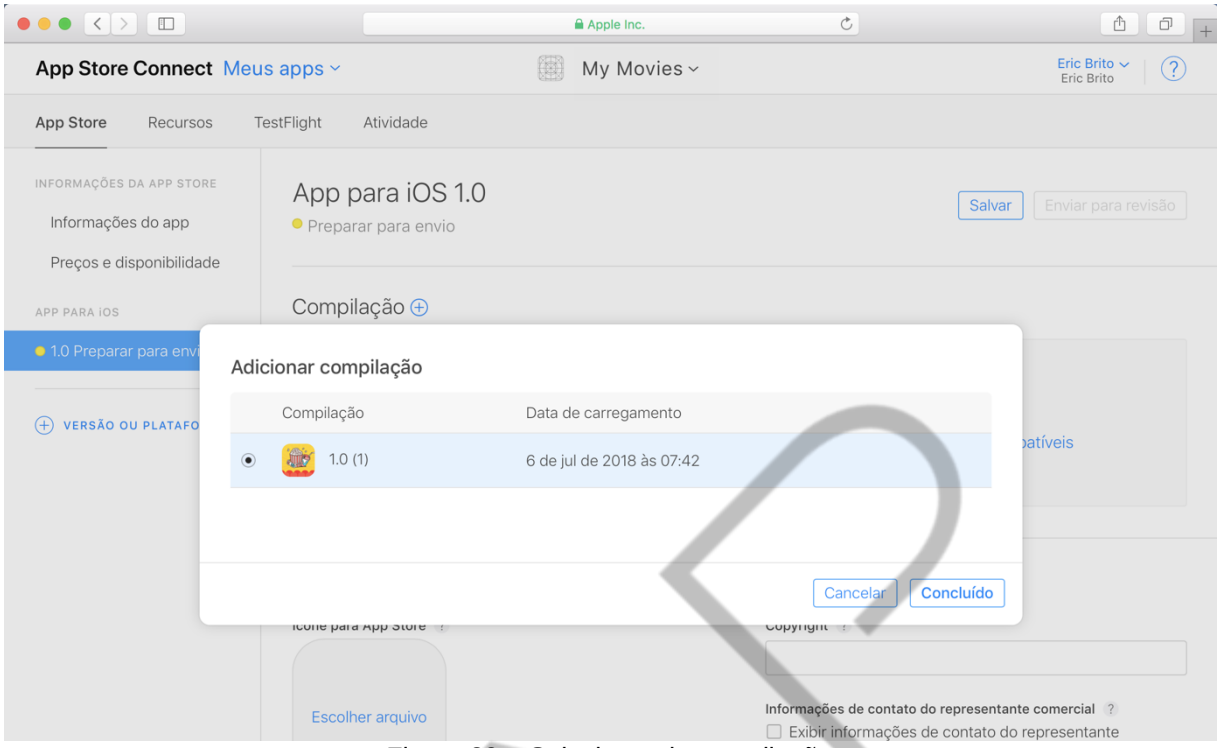

Figura 29 – Selecionando compilação
Fonte: Elaborado pelo autor (2019)

Preencha todas as demais informações que estiverem faltando nesta tela. Atente-se para a seção **Informações** para a equipe de revisão dos apps, desmarcando a opção **Necessário iniciar sessão,** pois nosso aplicativo não precisa de login para funcionar. Clique em **Salvar**, no topo direito da página e, na sequência, clique em **Enviar para revisão**.

Caso alguma informação essencial não tenha sido preenchida, a página irá informar o que estiver faltando para que você preencha. Se todas as informações estiverem preenchidas corretamente, uma nova tela irá surgir, perguntando a respeito de algumas informações extras do seu App, como por exemplo, se ele utiliza algum tipo de criptografia, se apresenta conteúdo de terceiros (para fins de direitos autorais) e se utiliza Identificador de publicidade. No nosso caso, podemos marcar **Não** para as três opções e clicar em **Enviar**.

Parabéns, a partir desse momento seu aplicativo está na fila de aprovação da Apple, em que será avaliado para ver se está de acordo com todos os Guidelines de desenvolvimento requisitados pela Apple e, se tudo estiver OK, será publicado na App Store. Esse processo costuma levar cerca de três dias, podendo ser menos ou mais.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Caso o aplicativo não esteja de acordo com alguma das diretrizes da Apple, eles avisarão por e-mail como proceder para a correção e submissão de uma nova compilação para avaliação.

CONCLUSÃO

Com essa lição, você concluiu também a nossa trilha iOS! Nessa jornada, você aprendeu uma nova linguagem de programação (*Swift*) e criou aplicativos que envolvam todos os recursos que o iOS oferece ao usuário. Nesse capítulo, finalizamos a construção de uma aplicação dinâmica com os principais recursos multimídia. Se você construiu seu projeto em paralelo a esta trilha, você já tem uma versão pronta para publicar na *App Store.*

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

APPLE. **Framework UIKit** 2020. Disponível em: <https://developer.apple.com/documentation/uikit>. Acesso em: 5 jun. 2022.

APPLE. **Framework WKWebView** 2020. Disponível em: <https://developer.apple.com/documentation/webkit/wkwebview>. Acesso em: 5 jun. 2022.

APPLE. **Framework MapKit** 2020. Disponível em: <https://developer.apple.com/documentation/mapkit>. Acesso em: 5 jun. 2022.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com